

NAME

mbio – Format independent input/output library for swath mapping sonar data.

VERSION

Version 5.8

DESCRIPTION

MBIO (**M**ulti**B**eam **I**nterface/**O**utput) is a library of functions used for reading and writing swath mapping sonar data files. **MBIO** supports a large number of data formats associated with different institutions and different sonar systems. The purpose of **MBIO** is to allow users to write processing and display programs which are independent of particular data formats and to provide a standard approach to swath mapping sonar data i/o.

MB-SYSTEM AUTHORSHIP

David W. Caress
Monterey Bay Aquarium Research Institute
Christian do Santos Ferreira
Center for Marine Environmental Sciences (MARUM)
University of Bremen
Dale N. Chayes
Center for Coastal and Ocean Mapping
University of New Hampshire

DATA TERMINOLOGY

MBIO handles three types of swath mapping data: beam bathymetry, beam amplitude, and sidescan. Both amplitude and sidescan represent measures of backscatter strength. Beam amplitudes are backscatter values associated with the same preformed beams used to obtain bathymetry; **MBIO** assumes that a bathymetry value exists for each amplitude value and uses the bathymetry beam location for the amplitude. Sidescan is generally constructed with a higher spatial resolution than bathymetry, and carries its own location parameters. In the context of **MB-System** documentation, the discrete values of bathymetry and amplitude are referred to as "beams", and the discrete values of sidescan are referred to as "pixels". An additional difference between "beam" and "pixel" data involves data flagging. An array of "beamflags" is carried by **MBIO** functions which allows the bathymetry (and by extension the amplitude) data to be flagged as bad. The details of the beamflagging scheme are presented below.

OVERVIEW

MBIO opens and initializes sonar data files for reading and writing using the functions **mb_read_init** and **mb_write_init**, respectively. These functions return a pointer to a data structure including all relevant information about the opened file, the control parameters which determine how data is read or written, and the arrays used for processing the data as it is read or written. This pointer is then passed to the functions used for reading or writing. There is no limit on the number of files which may be opened for reading or writing at any given time in a program.

The **mb_read_init** and **mb_write_init** functions also return initial maximum numbers of bathymetry beams, amplitude beams, and sidescan pixels that can be used to allocate data storage arrays of the appropriate sizes. However, for some data formats there are no specified maximum numbers of beams and pixels, and so in general the required dimensions may increase as data are read. Applications must pass appropriately dimensioned arrays into data extraction routines such as **mb_read**, **mb_get**, and **mb_get_all**. In order to enable dynamic memory management of these application arrays, the application must first register each array by passing the array pointer location to the function **mb_register_array**.

Data files are closed using the function **mb_close**. All internal and registered arrays are deallocated as part of

closing the file.

When it comes to actually reading and writing swath mapping sonar data, **MBIO** has three levels of i/o functionality:

- 1: Simple reading of swath data files. The primary functions are:
 - mb_read()**
 - mb_get()**

The positions of individual beams and pixels are returned in longitude and latitude by **mb_read()** and in acrosstrack and alongtrack distances by **mb_get()**. Only a limited set of navigation information is returned. Comments are also returned. These functions can be used without any special include files or any knowledge of the actual data structures used by the data formats or **MBIO**.
- 2: Complete reading and writing of data structures containing all of the available information. Data records may be read or written without extracting any of the information, or the swath data may be passed with the data structure. Several functions exist to extract information from or insert information into the data structures; otherwise, special include files are required to make sense of the sonar-specific data structures passed by level 2 i/o functions. The basic read and write functions that only pass pointers to internal data structures are:
 - mb_read_ping()**
 - mb_write_ping()**

The read and write routines which also extract or insert information are:

 - mb_get_all()**
 - mb_put_all()**
 - mb_put_comment()**

The information extraction and insertion functions are:

 - mb_insert()**
 - mb_extract()**
 - mb_extract_lonlat()**
 - mb_extract_nav()**
 - mb_insert_nav()**
 - mb_extract_altitude()**
 - mb_insert_altitude()**
 - mb_ttimes()**
 - mb_copyrecord()**
- 3: Buffered reading and writing of data structures containing all of the available information. The primary functions are:
 - mb_buffer_init()**
 - mb_buffer_close()**
 - mb_buffer_load()**

```

mb_buffer_dump()
mb_buffer_get_kind()
mb_buffer_get_next_data()
mb_buffer_extract()
mb_buffer_insert()
mb_buffer_get_next_nav()
mb_buffer_extract_nav()
mb_buffer_insert_nav()

```

The level 1 **MBIO** functions allow users to read sonar data independent of format, with the limitation that only a limited set of navigation information is passed. Thus, some of the information contained in certain data formats (e.g. the "heave" value in Hydrosweep DS data) is not passed by **mb_read()** or **mb_get()**. In general, the level 1 functions are useful for applications such as graphics which require only the navigation and the depth and/or backscatter values.

The level 2 functions (**mb_get_all()** and **mb_put_all()**) read and write the complete data structures, translate the data to internal data structures associated with each of the supported sonar systems, and pass pointers to these internal data structures. Additional functions allow a variety of information to be extracted from or inserted into the data structures (e.g. **mb_extract()** and **mb_insert()**). Additional information may be accessed using special include files to decode the data structures. The great majority of processing programs use level 2 functions.

The level 3 functions provide buffered reading and writing which is useful for applications that generate output files and need access to multiple pings at a time. In addition to reading (**mb_buffer_load()**) and writing (**mb_buffer_dump()**), functions exist for extracting information from the buffer (**mb_buffer_extract()**) and inserting information into the buffer (**mb_buffer_insert()**).

MBIO supports swath data in a number of different formats, each specified by a unique id number. The function **mb_format()** determines if a format id is valid. A set of similar functions returns information about the specified format (e.g. **mb_format_description()**, **mb_format_system()**, **mb_format_description()**, **mb_format_dimensions()**, **mb_format_flags()**, **mb_format_source()**, **mb_format_beamwidth()**).

Some **MB-System** programs can process multiple data files specified in "datalist" files. Each line of a datalist file contains a file path and the corresponding **MBIO** format id. Datalist files can be recursive and can contain comments. The functions used to extract input swath data file paths from datalist files includes **mb_datalist_open()**, **mb_datalist_read()**, and **mb_datalist_close()**.

A number of other **MBIO** functions dealing with default values for important parameters, error messages, memory management, and time conversions also exist and are discussed below.

SUPPORTED SWATH SONAR SYSTEMS

Each swath mapping sonar system outputs a data stream which includes some values or parameters unique to that system. In general, a number of different data formats have come into use for data from each of the sonar systems; many of these formats include only a subset of the original data stream. Internally, **MBIO** recognizes which sonar system each data format is associated with and uses a data structure including the complete data stream for that sonar. Consequently, it is possible to read and write the complete data stream when using the level 2 or 3 **MBIO** functions. The sonars and other sensors associated with supported formats include:

```

SeaBeam "classic" 16 beam multibeam sonar
Hydrosweep DS 59 beam multibeam sonar
Hydrosweep MD 40 beam mid-depth multibeam sonar
SeaBeam 2000 multibeam sonar
SeaBeam 2112, 2120, and 2130 multibeam sonars

```

Simrad EM12, EM121, EM950, and EM1000 multibeam sonars
 Simrad EM120, EM300, EM1002, and EM3000 multibeam sonars
 Kongsberg EM122, EM302, EM710, EM2040
 Kongsberg EM124, EM304, EM712, EM2040
 Hawaii MR-1 shallow tow interferometric sonar
 ELAC Bottomchart 1180 and 1050 multibeam sonars
 ELAC/SeaBeam Bottomchart Mk2 1180 and 1050 multibeam sonars
 Reson Seabat 9001/9002 multibeam sonars
 Reson Seabat 8101 multibeam sonars
 Simrad/Mesotech SM2000 multibeam sonars
 WHOI DSL AMS-120 deep tow interferometric sonar AMS-60 interferometric sonar
 WASSP multibeams (old format)
 Reson 7125 multibeam
 Teledyne T20, T50 multibeams
 R2Sonic multibeam
 3D at Depth subsea lidars (SL-1, SL-2, WiSSL)

SUPPORTED FORMATS

The following swath mapping sonar data formats are supported in this version of **MBIO**:

MBIO Data Format ID: 11

Format name: MBF_SBSIOMRG

Informal Description: SIO merge Sea Beam

Attributes: Sea Beam, bathymetry, 16 beams, binary, uncentered,
SIO.

MBIO Data Format ID: 12

Format name: MBF_SBSIOCEN

Informal Description: SIO centered Sea Beam

Attributes: Sea Beam, bathymetry, 19 beams, binary, centered,
SIO.

MBIO Data Format ID: 13

Format name: MBF_SBSIOLSI

Informal Description: SIO LSI Sea Beam

Attributes: Sea Beam, bathymetry, 19 beams, binary, centered,
obsolete, SIO.

MBIO Data Format ID: 14

Format name: MBF_SBURICEN

Informal Description: URI Sea Beam

Attributes: Sea Beam, bathymetry, 19 beams, binary, centered,
URI.

MBIO Data Format ID: 15

Format name: MBF_SBURIVAX

Informal Description: URI Sea Beam from VAX

Attributes: Sea Beam, bathymetry, 19 beams, binary, centered,
VAX byte order, URI.

MBIO Data Format ID: 16

Format name: MBF_SBSIOSWB

Informal Description: SIO Swath-bathy SeaBeam

Attributes: Sea Beam, bathymetry, 19 beams, binary, centered,
SIO.

MBIO Data Format ID: 17

Format name: MBF_SBIFREMR

Informal Description: IFREMER Archive SeaBeam

Attributes: Sea Beam, bathymetry, 19 beams, ascii, centered,
IFREMER.

MBIO Data Format ID: 21

Format name: MBF_HSATLRAW

Informal Description: Raw Hydrosweep

Attributes: Hydrosweep DS, bathymetry and amplitude, 59 beams,
ascii, Atlas Elektronik.

MBIO Data Format ID: 22

Format name: MBF_HSLDEDMB

Informal Description: EDMB Hydrosweep

Attributes: Hydrosweep DS, bathymetry, 59 beams, binary, NRL.

MBIO Data Format ID: 23

Format name: MBF_HSURICEN

Informal Description: URI Hydrosweep

Attributes: Hydrosweep DS, 59 beams, bathymetry, binary, URI.

MBIO Data Format ID: 24

Format name: MBF_HSLDEOIH

Informal Description: L-DEO in-house binary Hydrosweep

Attributes: Hydrosweep DS, 59 beams, bathymetry and amplitude,
binary, centered, L-DEO.

MBIO Data Format ID: 25

Format name: MBF_HSURIVAX

Informal Description: URI Hydrosweep from VAX

Attributes: Hydrosweep DS, 59 beams, bathymetry, binary,
VAX byte order, URI.

MBIO Data Format ID: 26

Format name: MBF_HSUNKNWN

Informal Description: Unknown Hydrosweep

Attributes: Hydrosweep DS, bathymetry, 59 beams, ascii, unknown origin, SOPAC.

MBIO Data Format ID: 32

Format name: MBF_SB2000SB

Informal Description: SIO Swath-bathy SeaBeam 2000 format

Attributes: SeaBeam 2000, bathymetry, 121 beams,
binary, SIO.

MBIO Data Format ID: 33

Format name: MBF_SB2000SS

Informal Description: SIO Swath-bathy SeaBeam 2000 format

Attributes: SeaBeam 2000, sidescan,
1000 pixels for 4-bit sidescan,

2000 pixels for 12+-bit sidescan,
binary, SIO.

MBIO Data Format ID: 41

Format name: MBF_SB2100RW

Informal Description: SeaBeam 2100 series vender format

Attributes: SeaBeam 2100, bathymetry, amplitude
and sidescan, 151 beams and 2000 pixels, ascii
with binary sidescan, SeaBeam Instruments.

MBIO Data Format ID: 42

Format name: MBF_SB2100B1

Informal Description: SeaBeam 2100 series vender format

Attributes: SeaBeam 2100, bathymetry, amplitude
and sidescan, 151 beams bathymetry,
2000 pixels sidescan, binary,
SeaBeam Instruments and L-DEO.

MBIO Data Format ID: 43

Format name: MBF_SB2100B2

Informal Description: SeaBeam 2100 series vender format

Attributes: SeaBeam 2100, bathymetry and amplitude,
151 beams bathymetry,
binary,
SeaBeam Instruments and L-DEO.

MBIO Data Format ID: 51

Format name: MBF_EMOLDRAW

Informal Description: Old Simrad vendor multibeam format

Attributes: Simrad EM1000, EM12S, EM12D,
and EM121 multibeam sonars,
bathymetry, amplitude, and sidescan,
60 beams for EM1000, 81 beams for EM12S/D,
121 beams for EM121, variable pixels,
ascii + binary, Simrad.

MBIO Data Format ID: 53

Format name: MBF_EM12IFRM

Informal Description: IFREMER TRISMUS format for Simrad EM12

Attributes: Simrad EM12S and EM12D,
bathymetry, amplitude, and sidescan
81 beams, variable pixels, binary,
read-only, IFREMER.

MBIO Data Format ID: 54

Format name: MBF_EM12DARW

Informal Description: Simrad EM12S RRS Darwin processed format

Attributes: Simrad EM12S, bathymetry and amplitude,
81 beams, binary, Oxford University.

MBIO Data Format ID: 56

Format name: MBF_EM300RAW

Informal Description: Simrad current multibeam vendor format

Attributes: Simrad EM120, EM300, EM1002, EM3000,
bathymetry, amplitude, and sidescan,
up to 254 beams, variable pixels, ascii + binary, Simrad.

MBIO Data Format ID: 57

Format name: MBF_EM300MBA

Informal Description: Simrad multibeam processing format

Attributes: Old and new Simrad multibeams,
EM12S, EM12D, EM121, EM120, EM300,
EM100, EM1000, EM950, EM1002, EM3000,
bathymetry, amplitude, and sidescan,
up to 254 beams, variable pixels, ascii + binary, MBARI.

MBIO Data Format ID: 58

Format name: MBF_EM710RAW

Informal Description: Kongsberg 3rd generation multibeam vendor format

Attributes: Kongsberg EM122, EM302, EM710,
bathymetry, amplitude, and sidescan,
up to 400 beams, variable pixels, binary, Kongsberg.

MBIO Data Format ID: 59

Format name: MBF_EM710MBA

Informal Description: Kongsberg current multibeam processing format

Attributes: Kongsberg EM122, EM302, EM710,
bathymetry, amplitude, and sidescan,
up to 400 beams, variable pixels, binary, MBARI.

MBIO Data Format ID: 61

Format name: MBF_MR1PRHIG

Informal Description: Obsolete SOEST MR1 post processed format

Attributes: SOEST MR1, bathymetry and sidescan,
variable beams and pixels, xdr binary,
SOEST, University of Hawaii.

MBIO Data Format ID: 62

Format name: MBF_MR1ALDEO

Informal Description: L-DEO MR1 post processed format with travel times

Attributes: L-DEO MR1, bathymetry and sidescan,
variable beams and pixels, xdr binary,
L-DEO.

MBIO Data Format ID: 63

Format name: MBF_MR1BLDEO

Informal Description: L-DEO small MR1 post processed format with travel times

Attributes: L-DEO MR1, bathymetry and sidescan,
variable beams and pixels, xdr binary,
L-DEO.

MBIO Data Format ID: 64

Format name: MBF_MR1PRVR2

Informal Description: SOEST MR1 post processed format

Attributes: SOEST MR1, bathymetry and sidescan,
variable beams and pixels, xdr binary,

SOEST, University of Hawaii.

MBIO Data Format ID: 71

Format name: MBF_MBLDEOIH

Informal Description: L-DEO in-house generic multibeam

Attributes: Data from all sonar systems, bathymetry,
amplitude and sidescan, variable beams and pixels,
binary, centered, L-DEO.

MBIO Data Format ID: 72

Format name: MBF_MBARIMB1

Informal Description: MBARI TRN swath bathymetry

Attributes: Downsampled bathymetry from multibeam sonars,
bathymetry only, variable beams, binary, MBARI

MBIO Data Format ID: 75

Format name: MBF_MBNETCDF

Informal Description: CARAIBES CDF multibeam

Attributes: Data from all sonar systems, bathymetry only,
variable beams, netCDF, IFREMER.

MBIO Data Format ID: 76

Format name: MBF_MBNCDFXT

Informal Description: CARAIBES CDF multibeam extended

Attributes: Superset of MBF_MBNETCDF, includes (at least SIMRAD EM12) amplitude,
variable beams, netCDF, IFREMER.

MBIO Data Format ID: 81

Format name: MBF_CBAT9001

Informal Description: Reson SeaBat 9001 shallow water multibeam

Attributes: 60 beam bathymetry and amplitude,
binary, University of New Brunswick.

MBIO Data Format ID: 82

Format name: MBF_CBAT8101

Informal Description: Reson SeaBat 8101 shallow water multibeam

Attributes: 101 beam bathymetry and amplitude,
binary, SeaBeam Instruments.

MBIO Data Format ID: 83

Format name: MBF_HYPC8101

Informal Description: Reson SeaBat 8101 shallow water multibeam

Attributes: 101 beam bathymetry,
ASCII, read-only, Coastal Oceanographics.

MBIO Data Format ID: 84

Format name: MBF_XTFR8101

Informal Description: XTF format Reson SeaBat 81XX

Attributes: 240 beam bathymetry and amplitude,
1024 pixel sidescan
binary, read-only,
Triton-Elis.

MBIO Data Format ID: 88

Format name: MBF_RESON7KR

Informal Description: Reson 7K multibeam vendor format

Attributes: Reson 7K series multibeam sonars,
bathymetry, amplitude, three channels sidescan, and subbottom
up to 254 beams, variable pixels, binary, Reson.

MBIO Data Format ID: 89

Format name: MBF_RESON7K3

Informal Description: Reson 7K multibeam vendor format

Attributes: Reson 7K series multibeam sonars,
bathymetry, amplitude, three channels sidescan, and subbottom
up to 254 beams, variable pixels, binary, Reson.

MBIO Data Format ID: 91

Format name: MBF_BCHRTUNB

Informal Description: Elac BottomChart shallow water multibeam

Attributes: 56 beam bathymetry and amplitude,
binary, University of New Brunswick.

MBIO Data Format ID: 92

Format name: MBF_ELMK2UNB

Informal Description: Elac BottomChart MkII shallow water multibeam

Attributes: 126 beam bathymetry and amplitude,
binary, University of New Brunswick.

MBIO Data Format ID: 93

Format name: MBF_BCHRXUNB

Informal Description: Elac BottomChart shallow water multibeam

Attributes: 56 beam bathymetry and amplitude,
binary, University of New Brunswick.

MBIO Data Format ID: 94

Format name: MBF_L3XSERAW

Informal Description: ELAC/SeaBeam XSE vendor format

Attributes: Bottomchart MkII 50 kHz and 180 kHz multibeam,
SeaBeam 2120 20 KHz multibeam,
bathymetry, amplitude and sidescan,
variable beams and pixels, binary,
L3 Communications (Elac Nautik
and SeaBeam Instruments).

MBIO Data Format ID: 101

Format name: MBF_HSMDARAW

Informal Description: Atlas HSMD medium depth multibeam raw format

Attributes: 40 beam bathymetry, 160 pixel sidescan,
XDR (binary), STN Atlas Elektronik.

MBIO Data Format ID: 102

Format name: MBF_HSMDLDIH

Informal Description: Atlas HSMD medium depth multibeam processed format

Attributes: 40 beam bathymetry, 160 pixel sidescan,
XDR (binary), L-DEO.

MBIO Data Format ID: 111

Format name: MBF_DSL120PF

Informal Description: WHOI DSL AMS-120 processed format

Attributes: 2048 beam bathymetry, 8192 pixel sidescan,
binary, parallel bathymetry and amplitude files, WHOI DSL.

MBIO Data Format ID: 112

Format name: MBF_DSL120SF

Informal Description: WHOI DSL AMS-120 processed format

Attributes: 2048 beam bathymetry, 8192 pixel sidescan,
binary, single files, WHOI DSL.

MBIO Data Format ID: 121

Format name: MBF_GSFGENMB

Informal Description: Leidos Generic Sensor Format (GSF) version GSF-v03.11

Attributes: variable beams, bathymetry and amplitude,
binary, single files, Leidos (formerly SAIC).

MBIO Data Format ID: 131

Format name: MBF_MSTIFFSS

Informal Description: MSTIFF sidescan format

Attributes: variable pixels, sidescan,
binary TIFF variant, single files, Sea Scan.

MBIO Data Format ID: 132

Format name: MBF_EDGJSTAR

Informal Description: Edgetech Jstar format

Attributes: variable pixels, dual frequency sidescan and subbottom,
binary SEG Y variant, single files,
low frequency sidescan returned as
survey data, Edgetech.

MBIO Data Format ID: 133

Format name: MBF_EDGJSTR2

Informal Description: Edgetech Jstar format

Attributes: variable pixels, dual frequency sidescan and subbottom,
binary SEG Y variant, single files,
high frequency sidescan returned as
survey data, Edgetech.

MBIO Data Format ID: 141

Format name: MBF_OICGEODA

Informal Description: OIC swath sonar format

Attributes: variable beam bathymetry and
amplitude, variable pixel sidescan, binary,
Oceanic Imaging Consultants

MBIO Data Format ID: 142

Format name: MBF_OICMBARI

Informal Description: OIC-style extended swath sonar format

Attributes: variable beam bathymetry and
amplitude, variable pixel sidescan, binary,
MBARI

MBIO Data Format ID: 151

Format name: MBF_OMGHDCSJ

Informal Description: UNB OMG HDCS format (the John Hughes Clarke format)

Attributes: variable beam bathymetry and
amplitude, variable pixel sidescan, binary,
UNB

MBIO Data Format ID: 160

Format name: MBF_SEGYSEGY

Informal Description: SEG Y seismic data format

Attributes: seismic or subbottom trace data,
single beam bathymetry, nav,
binary, SEG (SIOSEIS variant)

MBIO Data Format ID: 161

Format name: MBF_MGD77DAT

Informal Description: NGDC MGD77 underway geophysics format

Attributes: single beam bathymetry, nav, magnetics,
gravity, ascii, NOAA NGDC

MBIO Data Format ID: 162

Format name: MBF_ASCIIXYZ

Informal Description: Generic XYZ sounding format

Attributes: XYZ (lon lat depth) ASCII soundings, generic

MBIO Data Format ID: 163

Format name: MBF_ASCIIYZX

Informal Description: Generic YXZ sounding format

Attributes: YXZ (lat lon depth) ASCII soundings, generic

MBIO Data Format ID: 164

Format name: MBF_HYDROB93

Informal Description: NGDC binary hydrographic sounding format

Attributes: XYZ (lon lat depth) binary soundings

MBIO Data Format ID: 165

Format name: MBF_MBARIROV

Informal Description: MBARI ROV navigation format

Attributes: ROV navigation, MBARI

MBIO Data Format ID: 166

Format name: MBF_MBPRONAV

Informal Description: MB-System simple navigation format

Attributes: navigation, MBARI

MBIO Data Format ID: 167

Format name: MBF_NVNETCDF

Informal Description: CARAIBES CDF navigation

Attributes: netCDF, IFREMER.

MBIO Data Format ID: 168

Format name: MBF_ASCIIXYT

Informal Description: Generic XYT sounding format

Attributes: XYT (lon lat topography) ASCII soundings, generic

MBIO Data Format ID: 169

Format name: MBF_ASCIIYXT

Informal Description: Generic YXT sounding format

Attributes: YXT (lat lon topography) ASCII soundings, generic

MBIO Data Format ID: 170

Format name: MBF_MBARROV2

Informal Description: MBARI ROV navigation format

Attributes: ROV navigation, MBARI

MBIO Data Format ID: 171

Format name: MBF_HS10JAMS

Informal Description: Furuno HS-10 multibeam format,

Attributes: 45 beams bathymetry and amplitude,
ascii, JAMSTEC

MBIO Data Format ID: 172

Format name: MBF_HIR2RNAV

Informal Description: SIO GDC R2R navigation format

Attributes: R2R navigation, ascii, SIO

MBIO Data Format ID: 173

Format name: MBF_MGD77TXT

Informal Description: NGDC MGD77 underway geophysics format

Attributes: single beam bathymetry, nav, magnetics, gravity,
122 byte ascii records with CRLF line breaks, NOAA NGDC

MBIO Data Format ID: 174

Format name: MBF_MGD77TAB

Informal Description: NGDC MGD77 underway geophysics format

Attributes: single beam bathymetry, nav, magnetics, gravity,
122 byte ascii records with CRLF line breaks, NOAA NGDC

MBIO Data Format ID: 175

Format name: MBF_SOIUSBLN

Informal Description: SOI USBL navigation format

Attributes: SOI navigation, ascii, Schmidt Ocean Institute

MBIO Data Format ID: 176

Format name: MBF_SOIROVNV

Informal Description: SOI ROV navigation format(s)

Attributes: SOI navigation, ascii, Schmidt Ocean Institute

MBIO Data Format ID: 181

Format name: MBF_SAMESURF

Informal Description: SAM Electronics SURF format.

Attributes: variable beams, bathymetry, amplitude, and sidescan,
binary, single files, SAM Electronics (formerly Krupp-Atlas Elektronik).

MBIO Data Format ID: 182

Format name: MBF_HSDS2RAW

Informal Description: STN Atlas raw multibeam format

Attributes: STN Atlas multibeam sonars,
Hydrosweep DS2, Hydrosweep MD,
Fansweep 10, Fansweep 20,
bathymetry, amplitude, and sidescan,
up to 1440 beams and 4096 pixels,
XDR binary, STN Atlas.

MBIO Data Format ID: 183

Format name: MBF_HSDS2LAM

Informal Description: L-DEO HSDS2 processing format

Attributes: STN Atlas multibeam sonars,
Hydrosweep DS2, Hydrosweep MD,
Fansweep 10, Fansweep 20,
bathymetry, amplitude, and sidescan,
up to 1440 beams and 4096 pixels,
XDR binary, L-DEO.

MBIO Data Format ID: 191

Format name: MBF_IMAGE83P

Informal Description: Imagenex DeltaT Multibeam

Attributes: Multibeam, bathymetry, 480 beams, ascii + binary, Imagenex.

MBIO Data Format ID: 192

Format name: MBF_IMAGEMBA

Informal Description: MBARI DeltaT Multibeam

Attributes: Multibeam, bathymetry, 480 beams, ascii + binary, MBARI.

MBIO Data Format ID: 201

Format name: MBF_HYSWEEP1

Informal Description: HYSWEEP multibeam data format

Attributes: Many multibeam sonars,
bathymetry, amplitude
variable beams, ascii, HYPACK.

MBIO Data Format ID: 211

Format name: MBF_XTFB1624

Informal Description: XTF format Benthos Sidescan SIS1624

Attributes: variable pixels, dual frequency sidescan and subbottom,
xtf variant, single files,
low frequency sidescan returned as
survey data, Benthos.

MBIO Data Format ID: 221

Format name: MBF_SWPLSSXI

Informal Description: SEA interferometric sonar vendor intermediate format

Attributes: SEA SWATHplus,
bathymetry and amplitude,
variable beams, binary, SEA.

MBIO Data Format ID: 222

Format name: MBF_SWPLSSXP

Informal Description: SEA interferometric sonar vendor processed data format

Attributes: SEA SWATHplus,
bathymetry and amplitude,
variable beams, binary, SEA.

MBIO Data Format ID: 231

Format name: MBF_3DDEPTH

Informal Description: 3DatDepth prototype binary swath mapping LIDAR format

Attributes: 3DatDepth LIDAR, variable pulses, bathymetry and amplitude,
binary, 3DatDepth.

MBIO Data Format ID: 232

Format name: MBF_3DWISSLR

Informal Description: 3D at Depth Wide Swath Subsea Lidar (WiSSL) raw format

Attributes: 3D at Depth lidar, variable pulses, bathymetry and amplitude,
binary, 3D at Depth.

MBIO Data Format ID: 233

Format name: MBF_3DWISSLP

Informal Description: 3D at Depth Wide Swath Subsea Lidar (WiSSL) processing format

Attributes: 3D at Depth lidar, variable pulses, bathymetry and amplitude,
binary, MBARI.

MBIO Data Format ID: 234

Format name: MBF_3DWISSL2

Informal Description: 3D at Depth Second Generation Wide Swath Subsea Lidar (WiSSL2) SRIAT format

Attributes: 3D at Depth lidar, variable pulses, bathymetry and amplitude,
binary, MBARI.

MBIO Data Format ID: 241

Format name: MBF_WASSPENL

Informal Description: WASSP Multibeam Vendor Format

Attributes: WASSP multibeams,
bathymetry and amplitude,
122 or 244 beams, binary, Electronic Navigation Ltd.

MBIO Data Format ID: 251

Format name: MBF_PHOTGRAM

Informal Description: Stereo Photogrammetry format

Attributes: Stereo camera rigs, bathymetry, variable
soundings, binary, MBARI.

MBIO Data Format ID: 261

Format name: MBF_KEMKMALL

Informal Description: Kongsberg multibeam echosounder system kmall datagram format

Attributes: Kongsberg fourth generation multibeam sonars (EM2040, EM712,
EM304, EM124), bathymetry, amplitude, backscatter, variable beams,
binary datagrams, Kongsberg.

The institutional acronyms used above have the following meanings:

L-DEO Lamont-Doherty Earth Observatory
MBARI Monterey Bay Aquarium Research Institute
SIO Scripps Institution of Oceanography
WHOI Woods Hole Oceanographic Institution

URI University of Rhode Island
 NRL Naval Research Laboratory
 UNB University of New Brunswick
 UH University of Hawaii
 NOAA National Oceans and Atmospheres Agency
 NGDC National Geophysical Data Center
 USGS United States Geological Survey
 IFREMER French government agency responsible
 for operation of French oceanographic
 research fleet.

FUNCTION STATUS AND ERROR CODES

All of the **MBIO** functions return an integer status value with the convention that:

status = 1: success
 status = 0: failure

All **MBIO** functions also pass an error value argument which gives somewhat more information about problems than the status value. The full suite of possible error values and the associated error messages are:

error = 0: "No error",
 error = -1: "Time gap in data",
 error = -2: "Data outside specified location
 bounds",
 error = -3: "Data outside specified time interval",
 error = -4: "Ship speed too small",
 error = -5: "Comment record",
 error = -6: "Neither a data record nor a comment
 record",
 error = -7: "Unintelligible data record",
 error = -8: "Ignore this data",
 error = -9: "No data requested for buffer load",
 error = -10: "Data buffer is full",
 error = -11: "No data was loaded into the buffer",
 error = -12: "Data buffer is empty",
 error = -13: "No data was dumped from the buffer"
 error = -14: "No more survey data records in buffer"
 error = -15: "Data inconsistencies prevented
 inserting data into storage structure"
 error = 1: "Unable to allocate memory,
 initialization failed",
 error = 2: "Unable to open file,
 initialization failed",
 error = 3: "Illegal format identifier,
 initialization failed",
 error = 4: "Read error, probably end-of-file",
 error = 5: "Write error",
 error = 6: "No data in specified location bounds",
 error = 7: "No data in specified time interval",
 error = 8: "Invalid MBIO descriptor",
 error = 9: "Inconsistent usage of MBIO descriptor",
 error = 10: "No pings binned but no fatal error
 - this should not happen!",
 error = 11: "Invalid data record type specified
 for writing",
 error = 12: "Invalid control parameter specified

```

                                by user",
error = 13:    "Invalid buffer id",
error = 14:    "Invalid system id – this should
                                not happen!"
error = 15:    "This data file is not in the specified format!"

```

In general, programs should treat negative error values as non-fatal (reading and writing can continue) and positive error values as fatal (the data files should be closed and the program terminated).

FUNCTION VERBOSITY

All of the **MBIO** functions are passed a *verbose* parameter which controls how much debugging information is output to standard error. If *verbose* is 0 or 1, the **MBIO** functions will be silent. If *verbose* is 2, then each function will output information as it is entered and as it returns, along with the parameter values passed into and returned out of the function. Greater values of *verbose* will cause additional information to be output, including values at various stages of data processing during read and write operations. In general, programs using **MBIO** functions should adopt the following verbosity conventions:

```

verbose = 0:    "silent" or near-"silent" execution
verbose = 1:    simple output including
                  program name, version
                  and simple progress updates
verbose >= 2:  debug mode with copious output
                  including every function call
                  and status listings

```

INITIALIZATION AND CLOSING FUNCTIONS

```

int mb_read_init(
    int verbose,
    char *file,
    int format,
    int pings,
    int lonflip,
    double bounds[4],
    int btime_i[7],
    int etime_i[7],
    double speedmin,
    double timegap,
    void **mbio_ptr,
    double *btime_d,
    double *etime_d,
    int *beams_bath,
    int *beams_amp,
    int *pixels_ss,
    int *error);

```

The function **mb_read_init** initializes the data file to be read and the data structures required for reading the data. The *verbose* value controls the standard error output verbosity of the function.

The input control parameters have the following significance:

```

file:          input filename
format:        input MBIO data format id
pings:         ping averaging
lonflip:       longitude flipping
bounds:        location bounds of acceptable data
btime_i:       beginning time of acceptable data
etime_i:       ending time of acceptable data

```


speedmin: minimum ship speed of acceptable data
timegap: maximum time allowed before data gap

The format identifier *format* specifies which of the supported data formats is being read or written; the currently supported formats are listed in the "SUPPORTED FORMATS" section.

The *pings* parameter determines whether and how pings are averaged as part of data input. This parameter is used only by the functions **mb_read** and **mb_get**; **mb_get_all** and **mb_buffer_load** do not average pings. If *pings* = 1, then no ping averaging will be done and each ping read will be returned unaltered by the reading function. If *pings* > 1, then the navigation and beam data for *pings* pings will be read, averaged, and returned as the data for a single ping. If *pings* = 0, then the ping averaging will be varied so that the along-track distance between averaged pings is as close as possible to the across-track distance between beams.

The *lonflip* parameter determines the range in which longitude values are returned:

lonflip = -1 : -360 to 0
lonflip = 0 : -180 to 180
lonflip = 1 : 0 to 360

The *bounds* array sets the area within which data are desired. Data which lie outside the area specified by *bounds* will be returned with an error by the reading function. The functions **mb_read**, **mb_get** and **mb_get_all** use the *bounds* array; the function **mb_buffer_load** does no location checking.

bounds[0] : minimum longitude
bounds[1] : maximum longitude
bounds[2] : minimum latitude
bounds[3] : maximum latitude

The *btime_i* array sets the desired beginning time for the data and the *etime_i* array sets the desired ending time. If the beginning time is earlier than the ending time, then any data with a time stamp before the beginning time or after the ending time will be returned with an MB_ERROR_OUT_TIME error by the reading function. If the beginning time is after the ending time, then data with time stamps between the ending and beginning time are returned with an error. This scheme allows time windowing outside or inside a specified interval. The functions **mb_read**, **mb_get** and **mb_get_all** use the *btime_i* and *etime_i* arrays; the function **mb_buffer_load** does no time checking.

btime[0] : year
btime[1] : month
btime[2] : day
btime[3] : hour
btime[4] : minute
btime[5] : second
btime[6] : microsecond
etime[0] : year
etime[1] : month
etime[2] : day
etime[3] : hour
etime[4] : minute
etime[5] : second
etime[6] : microsecond

The *speedmin* parameter sets the minimum acceptable ship speed for the data. If the ship speed associated with any ping is less than *speedmin*, then that data will be returned with an error by the reading function. This is used to eliminate data collected while a ship is on station in a simple way. The functions **mb_read**, **mb_get** and **mb_get_all** use the *speedmin* value; the function **mb_buffer_load** does no speed checking.

The *timegap* parameter sets the threshold at which the time interval between sonar pings (or lidar scans, etc.) is regarded as a data gap. Swath data with ping intervals less than *timegap* are regarded as continuous; if the ping interval exceeds *timegap* then a data gap is declared. Ping averaging is not done across data gaps; an error is returned when time gaps are encountered. The functions **mb_read** and **mb_get** use the *timegap* value; the functions **mb_get_all** and **mb_buffer_load** do no ping averaging and thus have no need to check for time gaps.

The returned values are:

<i>mbio_ptr</i> :	pointer to an MBIO descriptor structure
<i>btime_d</i> :	desired beginning time in seconds since 1/1/70 00:00:0
<i>etime_d</i> :	desired ending time in seconds since 1/1/70 00:00:0
<i>beams_bath</i> :	maximum number of bathymetry beams
<i>beams_amp</i> :	maximum number of amplitude beams
<i>pixels_ss</i> :	maximum number of sidescan pixels
<i>error</i> :	error value

The structure pointed to by *mbio_ptr* holds the file descriptor and all of the control parameters which govern how the data is read; this pointer must be provided to the functions **mb_read**, **mb_get**, **mb_get_all**, or **mb_buffer_load** to read data. The values *beams_bath*, *beams_amp*, and *pixels_ss* return initial estimates of the maximum number of bathymetry and amplitude beams and sidescan pixels, respectively, that the specified data format may contain. In general, *beams_amp* will either be zero or equal to *beams_bath*. The values *btime_d* and *etime_d* give the desired beginning and end times of the data converted to seconds since 00:00:00 on January 1, 1970; **MBIO** uses these units to calculate time internally.

For most data formats, the initial maximum beam and pixel dimensions will not change. However, a few formats support both variable and arbitrarily large numbers of beams and/or pixels, and so applications must be capable of handling dynamic changes in the numbers of beams and pixels. The arrays allocated internally in the *mbio_ptr* structure are automatically increased when necessary. However, in order to successfully extract swath data using *mb_get*, *mb_get_all*, *mb_read*, or *mb_extract*, an application must also provide pointers to arrays large enough to hold the current maximum numbers of bathymetry beams, amplitude beams, and sidescan pixels. The function *mb_register_array* allows applications to register array pointers so that these arrays are also dynamically allocated by **MBIO**. Registered arrays will be managed as data are read and then freed when **mb_close** is called.

A status value indicating success or failure is returned; an error value argument passes more detailed information about initialization failures.

```
int mb_read_init_altnav(
    int verbose,
    char *file,
    int format,
    int pings,
    int lonflip,
    double bounds[4],
    int btime_i[7],
    int etime_i[7],
    double speedmin,
    double timegap,
    int astatus,
    char *apath,
    void ***mbio_ptr,
```

```

double *btime_d,
double *etime_d,
int *beams_bath,
int *beams_amp,
int *pixels_ss,
int *error);

```

The function **mb_read_init_altnav** initializes a data file for reading in exactly the same manner as **mb_read_init**, except that it also arranges for navigation to be drawn from an alternative navigation file *apath* rather than (or in addition to) the navigation embedded in *file* itself, where *astatus* (**MB_ALTNAV_USE** or **MB_ALTNAV_NONE**) indicates whether the alternative navigation should be used (see **mb_datalist_read3**, which supplies these values when reading a datalist).

```

-----
int mb_input_init(
    int verbose,
    char *socket_definition,
    int format,
    int pings,
    int lonflip,
    double bounds[4],
    int btime_i[7],
    int etime_i[7],
    double speedmin,
    double timegap,
    void **mbio_ptr,
    double *btime_d,
    double *etime_d,
    int *beams_bath,
    int *beams_amp,
    int *pixels_ss,
    int (*input_open)(int verbose, void *mbio_ptr, char *definition, int *error),
    int (*input_read)(int verbose, void *mbio_ptr, size_t *size, char *buffer, int *error),
    int (*input_close)(int verbose, void *mbio_ptr, int *error),
    int *error);

```

The function **mb_input_init** initializes an **MBIO** descriptor for reading in the same manner as **mb_read_init**, except that the data source is a socket, pipe, or other streamed input identified by *socket_definition* rather than an ordinary file, and the calling program supplies its own *input_open*, *input_read*, and *input_close* callback functions to open, read raw bytes from, and close that input source. This allows **MBIO** to read swath data streamed in real time (e.g. **mbtrnpp**) using the same **mb_read/mb_get_all/mb_extract** interface used for ordinary files.

```

-----
int mb_set_debug_records(
    int verbose,
    void *mbio_ptr,
    bool enable_debug_record_type_listing,
    int num_debug_record_identifiers,
    mb_name *debug_record_identifiers,
    int *error);

```

The function **mb_set_debug_records** configures the **MBIO** descriptor pointed to by *mbio_ptr* to print a

debug listing of selected data record types as they are read. If *enable_debug_record_type_listing* is true, the *num_debug_record_identifiers* record type names in the array *debug_record_identifiers* are the ones listed; this is a debugging aid for programs (e.g. **mbtrnpp**) that process streamed input.

```
-----
int mb_set_platform(
    int verbose,
    void *mbio_ptr,
    void *mbplatform_ptr,
    int *error);
```

The function **mb_set_platform** attaches a previously initialized sensor platform model (see **mb_platform_init**) pointed to by *mbplatform_ptr* to the **MBIO** descriptor pointed to by *mbio_ptr*, so that subsequent reads through that descriptor can make use of the platform's sensor offsets.

```
-----
int mb_write_init(
    int verbose,
    char *file,
    int format,
    void **mbio_ptr,
    int *beams_bath,
    int *beams_amp,
    int *pixels_ss,
    int *error);
```

The function **mb_write_init** initializes the data file to be written and the data structures required for writing the data. The *verbose* value controls the standard error output verbosity of the function.

The input control parameters have the following significance:

<i>file</i> :	output filename
<i>format</i> :	output MBIO data format id

The returned values are:

<i>mbio_ptr</i> :	pointer to a structure describing the output file
<i>beams_bath</i> :	maximum number of bathymetry beams
<i>beams_back</i> :	maximum number of backscatter beams
<i>error</i> :	error value

The structure pointed to by *mbio_ptr* holds the output file descriptor; this pointer must be provided to the functions **mb_write**, **mb_put**, **mb_put_all**, or **mb_buffer_dump** to write data. The values *beams_bath*, *beams_amp*, and *pixels_ss* return the maximum number of bathymetry and amplitude beams and sidescan pixels, respectively, that the specified data format may contain. In general, *beams_amp* will either be zero or equal to *beams_bath*. In order to successfully write data, the calling program must provide pointers to arrays large enough to hold *beams_bath* bathymetry values, *beams_amp* amplitude values, and *pixels_ss* sidescan values.

For most data formats, the initial maximum beam and pixel dimensions will not change. However, a few formats support both variable and arbitrarily large numbers of beams and/or pixels, and so applications must be capable of handling dynamic changes in the numbers of beams and pixels. The arrays allocated internally in the *mbio_ptr* structure are automatically increased when necessary. However, in order to successfully insert modified swath data using *mb_put*, *mb_put_all*, or *mb_insert*, an application must also provide pointers to

arrays large enough to hold the current maximum numbers of bathymetry beams, amplitude beams, and sidescan pixels. The function `mb_register_array` allows applications to register array pointers so that these arrays are also dynamically allocated by *MBIO*. Registered arrays will be managed as data are read and written and then freed when `mb_close` is called.

A status value indicating success or failure is returned; an error value argument passes more detailed information about initialization failures.

```
-----
int mb_register_array(
    int verbose,
    void *mbio_ptr,
    int type,
    size_t size,
    void **handle,
    int *error);
```

Registers an array pointer `*handle` so that the size of the allocated array can be managed dynamically by **MBIO**. Note that the location `**handle` of the array pointer must be supplied, not the pointer value `*handle`. The pointer value `*handle` should initially be `NULL`. The type value indicates whether this array is to be dimensioned according to the maximum number of bathymetry beams (type = 1), amplitude beams (type = 2), or sidescan pixels (type = 3). The size value indicates the size of each element array in bytes (e.g. a char array has size = 1, a short array has size = 2, an int array or a float array have size = 4, and a double array has size = 8). The array is associated with the **MBIO** descriptor `mbio_ptr`, and is freed when `mb_close` is called for this particular `mbio_ptr`.

```
-----
int mb_close(
    int verbose,
    void **mbio_ptr,
    int *error);
```

Closes the data file listed in the **MBIO** descriptor pointed to by `*mbio_ptr` and releases all specially allocated memory, including all application arrays registered using `mb_register_array`. The pointer `*mbio_ptr` is deallocated and set to `NULL`. The verbose value controls the standard error output verbosity of the function. A status value indicating success or failure is returned; an error value argument passes more detailed information about failures.

```
-----
int mb_fileio_open(
    int verbose,
    void *mbio_ptr,
    int *error);
```

The function `mb_fileio_open` initializes low level byte input/output for a single regular file associated with the **MBIO** descriptor pointed to by `mbio_ptr`, and is called internally by `mb_read_init` and `mb_write_init`. Depending on the platform and data format, the underlying implementation may use buffered `fread()/fwrite()`, local buffering, or `mmap()`.

```
-----
int mb_fileio_close(
    int verbose,
    void *mbio_ptr,
```

```
int *error);
```

The function **mb_fileio_close** releases the resources allocated by **mb_fileio_open** for the **MBIO** descriptor pointed to by *mbio_ptr*, and is called internally by **mb_close**.

```
-----
int mb_fileio_get(
    int verbose,
    void *mbio_ptr,
    char *buffer,
    size_t *size,
    int *error);
```

The function **mb_fileio_get** reads **size* bytes from the input file associated with the **MBIO** descriptor pointed to by *mbio_ptr* into *buffer*.

```
-----
int mb_fileio_put(
    int verbose,
    void *mbio_ptr,
    char *buffer,
    size_t *size,
    int *error);
```

The function **mb_fileio_put** writes **size* bytes from *buffer* to the output file associated with the **MBIO** descriptor pointed to by *mbio_ptr*.

```
-----
int mb_copyfile(
    int verbose,
    const char *src,
    const char *dst,
    int *error);
```

The function **mb_copyfile** copies the file *src* to the file *dst*.

```
-----
int mb_catfiles(
    int verbose,
    const char *src1,
    const char *src2,
    const char *dst,
    int *error);
```

The function **mb_catfiles** concatenates the files *src1* and *src2*, in that order, into the file *dst*.

LEVEL 1 FUNCTIONS

```
-----
int mb_read(
    int verbose,
    void *mbio_ptr,
    int *kind,
    int *pings,
```

```

    int time_i[7],
    double *time_d,
    double *navlon,
    double *navlat,
    double *speed,
    double *heading,
    double *distance,
    double *altitude,
    double *sonardepth,
    int *nbath,
    int *namp,
    int *nss,
    char *beamflag,
    double *bath,
    double *amp,
    double *bathlon,
    double *bathlat,
    double *ss,
    double *sslon,
    double *sslat,
    char *comment,
    int *error);

```

The function **mb_read** reads, processes, and returns sonar data according to the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function. A number of different data record types are recognized by **MB-System**, but **mb_read()** only returns survey and comment data records. The *kind* value indicates which type of record has been read. The data is in the form of bathymetry, amplitude, and sidescan values combined with the longitude and latitude locations of the bathymetry and sidescan measurements (amplitudes are coincident with the bathymetry).

The return values are:

<i>kind:</i>	kind of data record read
	1 survey data
	2 comment
	>=3 other data that cannot be passed by mb_read
<i>pings:</i>	number of pings averaged to give current data
<i>time_i:</i>	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d:</i>	time of current ping in seconds since 1/1/70 00:00:00
<i>navlon:</i>	longitude
<i>navlat:</i>	latitude
<i>speed:</i>	ship speed in km/s
<i>heading:</i>	ship heading in degrees
<i>distance:</i>	distance along shiptrack since last ping in km

<i>altitude:</i>	altitude of sonar above seafloor in m
<i>sonardepth:</i>	depth of sonar in m
<i>nbath:</i>	number of bathymetry values
<i>namp:</i>	number of amplitude values
<i>nss:</i>	number of sidescan values
<i>beamflag:</i>	array of bathymetry flags
<i>bath:</i>	array of bathymetry values in meters
<i>amp:</i>	array of amplitude values in unknown units
<i>bathlon:</i>	array of longitude values corresponding to bathymetry
<i>bathlat:</i>	array of latitude values corresponding to bathymetry
<i>ss:</i>	array of sidescan values in unknown units
<i>sslon:</i>	array of longitude values corresponding to sidescan
<i>sslat:</i>	array of latitude values corresponding to sidescan
<i>comment:</i>	comment string
<i>error:</i>	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about read failures.

```
int mb_get(
    int verbose,
    void *mbio_ptr,
    int *kind,
    int *pings,
    int time_i[7],
    double *time_d,
    double *navlon,
    double *navlat,
    double *speed,
    double *heading,
    double *distance,
    double *altitude,
    double *sonardepth,
    int *nbath,
    int *namp,
    int *nss,
    char *beamflag,
    double *bath,
    double *amp,
    double *bathacrosstrack,
    double *bathalongtrack,
    double *ss,
    double *ssacrosstrack,
    double *ssalongtrack,
    char *comment,
    int *error);
```


The function **mb_get** reads, processes, and returns sonar data according to the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function. A number of different data record types are recognized by **MB-System**, but **mb_get()** only returns survey and comment data records. The *kind* value indicates which type of record has been read. The data is in the form of bathymetry, amplitude, and sidescan values combined with the acrosstrack and alongtrack distances relative to the navigation of the bathymetry and sidescan measurements (amplitudes are coincident with the bathymetry values).

The return values are:

<i>kind:</i>	kind of data record read
	1 survey data
	2 comment
	>=3 other data that cannot be passed by mb_get
<i>pings:</i>	number of pings averaged to give current data
<i>time_i:</i>	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d:</i>	time of current ping in seconds since 1/1/70 00:00:00
<i>navlon:</i>	longitude
<i>navlat:</i>	latitude
<i>speed:</i>	ship speed in km/s
<i>heading:</i>	ship heading in degrees
<i>distance:</i>	distance along shiptrack since last ping in km
<i>altitude:</i>	altitude of sonar above seafloor in m
<i>sonardepth:</i>	depth of sonar in m
<i>nbath:</i>	number of bathymetry values
<i>namp:</i>	number of amplitude values
<i>nss:</i>	number of sidescan values
<i>beamflag:</i>	array of bathymetry flags
<i>bath:</i>	array of bathymetry values in meters
<i>amp:</i>	array of amplitude values in unknown units
<i>bathacrosstrack:</i>	array of acrosstrack distances in meters corresponding to bathymetry
<i>bathalongtrack:</i>	array of alongtrack distances in meters corresponding to bathymetry
<i>ss:</i>	array of sidescan values in unknown units
<i>ssacrosstrack:</i>	array of acrosstrack distances in meters corresponding to sidescan
<i>ssalongtrack:</i>	array of alongtrack distances in meters corresponding to sidescan
<i>comment:</i>	comment string
<i>error:</i>	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about read failures.

LEVEL 2 FUNCTIONS

```
int mb_read_ping(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int *error);
```

The function **mb_read_ping** reads and returns sonar data according to the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function. The data is returned one record at a time; no averaging is performed. A pointer to a data structure containing all of the data read is returned as *store_ptr*; the form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**; the *kind* value indicates which type of record has been read.

The return values are:

<i>store_ptr</i> :	pointer to complete data structure
<i>kind</i> :	kind of data record read
	1 survey data
	2 comment
	3 header
	4 calibrate
	5 mean sound speed
	6 SVP
	7 standby
	8 nav source
	9 parameter
	10 start
	11 stop
	12 nav
	13 run parameter
	14 clock
	15 tide
	16 height
	17 heading
	18 attitude
	19 ssv
	20 angle
	21 event
	22 history
	23 summary
	24 processing parameters
	25 sensor parameters
	26 navigation error
	27 uninterpretable line
<i>error</i> :	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about read failures.

```
-----
int mb_write_ping(
    int verbose,
```

```
void *mbio_ptr,
void *store_ptr,
int *error);
```

The function **mb_write_ping** writes sonar data to the file listed in the **MBIO** descriptor pointed to by *MBIO_ptr*. The *verbose* value controls the standard error output verbosity of the function. A pointer to a data structure containing all of the data read is passed as *store_ptr*; the form of the data structure is determined by the sonar system associated with the format of the data being written. The values to be output are:

store_ptr: pointer to complete data structure

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about write failures.

```
int mb_get_store(
    int verbose,
    void *mbio_ptr,
    void **store_ptr,
    int *error);
```

The function **mb_get_store()** returns a pointer **store_ptr* to the data storage structure associated with a particular **MBIO** descriptor *mbio_ptr*. The **mb_read_init()** and **mb_write_init()** functions both allocate one of these internal storage structures. The form of the data structure is determined by the sonar system associated with the format of the data being written. Storage structure pointers must be passed to level two **MBIO** functions such as **mb_write_ping()** and **mb_insert()**. The *verbose* value controls the standard error output verbosity of the function.

The return values are:

store_ptr: pointer to complete data structure
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_dimensions(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int *nbath,
    int *namp,
    int *nss,
    int *error);
```

The function **mb_dimensions** returns the kind of the current data record along with the numbers of bathymetry, amplitude, and sidescan values contained in it, without extracting the values themselves. This is useful for sizing arrays before calling **mb_extract** or similar functions.

```
int mb_pingnumber(
    int verbose,
    void *mbio_ptr,
    unsigned int *pingnumber,
    int *error);
```

The function **mb_pingnumber** returns a ping number for the survey data record most recently read, taken directly from the data if the format records one, or otherwise set to the count of survey records read so far by this **MBIO** descriptor.

```
-----
int mb_segynumber(
    int verbose,
    void *mbio_ptr,
    unsigned int *line,
    unsigned int *shot,
    unsigned int *cdp,
    int *error);
```

The function **mb_segynumber** returns the seismic reflection line number, shot number, and common depth point (cdp) number associated with the current data record, for data formats that carry subbottom or seismic reflection data that can be represented in **SEG Y** form (see **mb_extract_segy** below). For formats that do not provide these values, **shot* is set to the count of survey records read so far and **line* and **cdp* are set to zero.

```
-----
int mb_beamwidths(
    int verbose,
    void *mbio_ptr,
    double *beamwidth_xtrack,
    double *beamwidth_ltrack,
    int *error);
```

The function **mb_beamwidths** returns the nominal athwartships and alongtrack sonar beam widths in degrees, as stored in the **MBIO** descriptor pointed to by *mbio_ptr* when the file was opened (see **mb_format_beamwidth**).

```
-----
int mb_sonartype(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *sonartype,
    int *error);
```

The function **mb_sonartype** returns a code (one of the **MB_TOPOGRAPHY_TYPE** values, e.g. multibeam, sidescan, interferometric, or unknown) indicating the general type of sonar that produced the current data record, where the data format and system can distinguish this.

```
-----
int mb_sidescantype(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
```

```

    int *ss_type,
    int *error);

```

The function **mb_sidescantype** returns a code (one of the **MB_SIDESCAN_TYPE** values) indicating whether the sidescan imagery associated with the current data record is referenced to a linear (uncorrected) or logarithmic (corrected) intensity scale, where the data format and system can distinguish this.

```

-----
int mb_get_all(
    int verbose,
    void *mbio_ptr,
    void **store_ptr,
    int *kind,
    int time_i[7],
    double *time_d,
    double *navlon,
    double *navlat,
    double *speed,
    double *heading,
    double *distance,
    double *altitude,
    double *sonardepth,
    int *nbath,
    int *namp,
    int *nss,
    char *beamflag,
    double *bath,
    double *amp,
    double *bathacrosstrack,
    double *bathalongtrack,
    double *ss,
    double *ssacrosstrack,
    double *ssalongtrack,
    char *comment,
    int *error);

```

The function **mb_get_all** reads and returns sonar data according to the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function. The data is returned one record at a time; no averaging is performed. A pointer to a data structure containing all of the data read is returned as *store_ptr*; the form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**; the *kind* value indicates which type of record has been read. Additional data is returned if the data record is survey data (navigation, bathymetry, amplitude, and sidescan), navigation data (navigation only), or comment data (comment only).

The return values are:

<i>store_ptr</i> :	pointer to complete data structure
<i>kind</i> :	kind of data record read
	1 survey data
	2 comment
	3 header
	4 calibrate
	5 mean sound speed

	6	SVP
	7	standby
	8	nav source
	9	parameter
	10	start
	11	stop
	12	nav
	13	run parameter
	14	clock
	15	tide
	16	height
	17	heading
	18	attitude
	19	ssv
	20	angle
	21	event
	22	history
	23	summary
	24	processing parameters
	25	sensor parameters
	26	navigation error
	27	uninterpretable line
<i>time_i:</i>	time of current ping	
	<i>time_i</i> [0]: year	
	<i>time_i</i> [1]: month	
	<i>time_i</i> [2]: day	
	<i>time_i</i> [3]: hour	
	<i>time_i</i> [4]: minute	
	<i>time_i</i> [5]: second	
	<i>time_i</i> [6]: microsecond	
<i>time_d:</i>	time of current ping in seconds	
	since 1/1/70 00:00:00	
<i>navlon:</i>	longitude	
<i>navlat:</i>	latitude	
<i>speed:</i>	ship speed in km/s	
<i>heading:</i>	ship heading in degrees	
<i>distance:</i>	distance along shiptrack since last ping in km	
<i>altitude:</i>	altitude of sonar above seafloor in m	
<i>sonardepth:</i>	depth of sonar in m	
<i>nbath:</i>	number of bathymetry values	
<i>namp:</i>	number of amplitude values	
<i>nss:</i>	number of sidescan values	
<i>beamflag:</i>	array of bathymetry flags	
<i>bath:</i>	array of bathymetry values in meters	
<i>amp:</i>	array of amplitude values in unknown units	
<i>bathacrosstrack:</i>	array of acrosstrack distances in meters corresponding to bathymetry	
<i>bathalongtrack:</i>	array of alongtrack distances in meters corresponding to bathymetry	
<i>ss:</i>	array of sidescan values in unknown units	
<i>ssacrosstrack:</i>	array of acrosstrack distances	

	in meters corresponding to sidescan
<i>ssacrosstrack</i> :	array of alongtrack distances
	in meters corresponding to sidescan
<i>comment</i> :	comment string
<i>error</i> :	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about read failures.

```
-----
int mb_put_all(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int usevalues,
    int kind,
    int time_i[7],
    double time_d,
    double navlon,
    double navlat,
    double speed,
    double heading,
    int nbath,
    int namp,
    int nss,
    char *beamflag,
    double *bath,
    double *amp,
    double *bathacrosstrack,
    double *bathalongtrack,
    double *ss,
    double *ssacrosstrack,
    double *ssalongtrack,
    char *comment,
    int *error);
```

The function **mb_put_all** writes sonar data to the file listed in the **MBIO** descriptor pointed to by *MBIO_ptr*. The *verbose* value controls the standard error output verbosity of the function. A pointer to a data structure containing all of the data read is passed as *store_ptr*; the form of the data structure is determined by the sonar system associated with the format of the data being written. Additional data is passed if the data record is survey data (navigation, bathymetry, amplitude, and sidescan), navigation data (navigation only), or comment data (comment only). If the *usevalues* flag is set to 1, then the passed values will be inserted in the data structure pointed to by *store_ptr* before the data is written. If the *usevalues* flag is set to 0, the data structure pointed to by *store_ptr* will be written without modification. The values to be output are:

<i>store_ptr</i> :	pointer to complete data structure
<i>usevalues</i> :	flag controlling use of data passed by value
	0 do not insert into data structure before writing the data
	1 insert into data structure before writing the data
<i>kind</i> :	kind of data record to be written
	1 survey data

	2	comment
	3	header
	4	calibrate
	5	mean sound speed
	6	SVP
	7	standby
	8	nav source
	9	parameter
	10	start
	11	stop
	12	nav
	13	run parameter
	14	clock
	15	tide
	16	height
	17	heading
	18	attitude
	19	ssv
	20	angle
	21	event
	22	history
	23	summary
	24	processing parameters
	25	sensor parameters
	26	navigation error
	27	uninterpretable line
<i>time_i</i> :	time of current ping (used if <i>time_i</i> [0] != 0)	
	<i>time_i</i> [0]: year	
	<i>time_i</i> [1]: month	
	<i>time_i</i> [2]: day	
	<i>time_i</i> [3]: hour	
	<i>time_i</i> [4]: minute	
	<i>time_i</i> [5]: second	
	<i>time_i</i> [6]: microsecond	
<i>time_d</i> :	time of current ping in seconds since	
	1/1/70 00:00:00 (used if <i>time_i</i> [0] = 0)	
<i>navlon</i> :	longitude	
<i>navlat</i> :	latitude	
<i>speed</i> :	ship speed in km/s	
<i>heading</i> :	ship heading in degrees	
<i>nbath</i> :	number of bathymetry values	
<i>namp</i> :	number of amplitude values	
<i>nss</i> :	number of sidescan values	
<i>beamflag</i> :	array of bathymetry flags	
<i>bath</i> :	array of bathymetry values in meters	
<i>amp</i> :	array of amplitude values in unknown units	
<i>bathacrosstrack</i> :	array of acrosstrack distances	
	in meters corresponding to bathymetry	
<i>bathalongtrack</i> :	array of alongtrack distances	
	in meters corresponding to bathymetry	
<i>ss</i> :	array of sidescan values in unknown units	
<i>ssacrosstrack</i> :	array of acrosstrack distances	
	in meters corresponding to sidescan	

ssacrosstrack: array of alongtrack distances
in meters corresponding to sidescan
comment: comment string

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about write failures.

```
int mb_put_comment(
    int verbose,
    void *mbio_ptr,
    char *comment,
    int *error);
```

The function **mb_put_comment** writes a comment to the file listed in the **MBIO** descriptor pointed to by *MBIO_ptr*. The *verbose* value controls the standard error output verbosity of the function. The data is in the form of a null terminated string. The maximum length of comments varies with different data formats. In general individual comments should be less than 80 characters long to insure compatibility with all formats. The values to be output are:

comment: comment string

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about write failures.

```
int mb_extract_platform(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    void **platform_ptr,
    int *error);
```

The function **mb_extract_platform** extracts sensor platform information (the sonar and auxiliary sensor positions and orientation offsets) embedded in the current data record, where the data format supports this, and returns it as a newly allocated platform structure pointed to by **platform_ptr* (see **mb_platform_init** below). This is used by **mb_preprocess** when a per-file or per-record platform definition is available directly from the raw data rather than from a separately supplied platform file.

```
int mb_sensorhead(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *sensorhead,
    int *error);
```

The function **mb_sensorhead** returns the index of the sensor head (transducer) that produced the current data record, for multi-head sonar systems (e.g. the Kraken/RESON WISSL laser system) where the data format distinguishes among heads.

```
-----
int mb_preprocess(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    void *platform_ptr,
    void *preprocess_pars_ptr,
    int *error);
```

The function **mb_preprocess** applies sensor offset, timing, and navigation/attitude merging corrections to the current data record read by *mbio_ptr*, using the platform model pointed to by *platform_ptr* (see **mb_platform_init**) and the options and merge navigation/attitude time series pointed to by *preprocess_pars_ptr* (a *struct mb_preprocess_struct*, filled in by the calling program, typically **mb_preprocess**). If the data format has registered its own preprocessing routine, that routine is called; otherwise **mb_preprocess_generic** is called so that platform-based preprocessing is available for every supported data format.

```
-----
int mb_preprocess_generic(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    void *platform_ptr,
    void *preprocess_pars_ptr,
    int *error);
```

The function **mb_preprocess_generic** implements the format-independent (generic) version of the preprocessing corrections applied by **mb_preprocess**, operating solely through the standard **MBIO mb_extract/mb_insert** family of functions. It is used automatically by **mb_preprocess** for any data format that has not registered a format-specific preprocessing routine of its own.

```
-----
int mb_extract(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int time_i[7],
    double *time_d,
    double *navlon,
    double *navlat,
    double *speed,
    double *heading,
    int *nbath,
    int *namp,
    int *nss,
    char *beamflag,
    double *bath,
    double *amp,
    double *bathacrosstrack,
```

```

double *bathalongtrack,
double *ss,
double *ssacrosstrack,
double *ssalongtrack,
char *comment,
int *error);

```

The function **mb_extract** extracts sonar data from the structure pointed to by **store_ptr* according to the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function. The form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**; the *kind* value indicates which type of record is stored in **store_ptr*. Additional data is returned if the data record is survey data (navigation, bathymetry, amplitude, and sidescan), navigation data (navigation only), or comment data (comment only).

The return values are:

<i>kind:</i>	kind of data record read
	1 survey data
	2 comment
	12 navigation
<i>time_i:</i>	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d:</i>	time of current ping in seconds
	since 1/1/70 00:00:00
<i>navlon:</i>	longitude
<i>navlat:</i>	latitude
<i>speed:</i>	ship speed in km/s
<i>heading:</i>	ship heading in degrees
<i>distance:</i>	distance along shiptrack since last
	ping in km
<i>altitude:</i>	altitude of sonar above seafloor
	in m
<i>sonardepth:</i>	depth of sonar in m
<i>nbath:</i>	number of bathymetry values
<i>namp:</i>	number of amplitude values
<i>nss:</i>	number of sidescan values
<i>beamflag:</i>	array of bathymetry flags
<i>bath:</i>	array of bathymetry values in meters
<i>amp:</i>	array of amplitude values in unknown units
<i>bathacrosstrack:</i>	array of acrosstrack distances
	in meters corresponding to bathymetry
<i>bathalongtrack:</i>	array of alongtrack distances
	in meters corresponding to bathymetry
<i>ss:</i>	array of sidescan values in unknown units
<i>ssacrosstrack:</i>	array of acrosstrack distances
	in meters corresponding to sidescan
<i>ssalongtrack:</i>	array of alongtrack distances

	in meters corresponding to sidescan
<i>comment:</i>	comment string
<i>error:</i>	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about extract failures.

```

-----
int mb_extract_lonlat(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int time_i[7],
    double *time_d,
    double *navlon,
    double *navlat,
    double *speed,
    double *heading,
    int *nbath,
    int *namp,
    int *nss,
    char *beamflag,
    double *bath,
    double *amp,
    double *bathlon,
    double *bathlat,
    double *ss,
    double *sslon,
    double *sslat,
    char *comment,
    int *error);

```

The function **mb_extract_lonlat** extracts sonar data from the structure pointed to by **store_ptr* according to the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function. The form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**; the *kind* value indicates which type of record is stored in **store_ptr*. Additional data is returned if the data record is survey data (navigation, bathymetry, amplitude, and sidescan), navigation data (navigation only), or comment data (comment only).

The return values are:

<i>kind:</i>	kind of data record read
	1 survey data
	2 comment
	12 navigation
<i>time_i:</i>	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second

	<i>time_i</i> [6]: microsecond
<i>time_d</i> :	time of current ping in seconds since 1/1/70 00:00:00
<i>navlon</i> :	longitude
<i>navlat</i> :	latitude
<i>speed</i> :	ship speed in km/s
<i>heading</i> :	ship heading in degrees
<i>distance</i> :	distance along shiptrack since last ping in km
<i>altitude</i> :	altitude of sonar above seafloor in m
<i>sonardepth</i> :	depth of sonar in m
<i>nbath</i> :	number of bathymetry values
<i>namp</i> :	number of amplitude values
<i>nss</i> :	number of sidescan values
<i>beamflag</i> :	array of bathymetry flags
<i>bath</i> :	array of bathymetry values in meters
<i>amp</i> :	array of amplitude values in unknown units
<i>bathlon</i> :	array of longitude positions in degrees corresponding to bathymetry
<i>bathlat</i> :	array of latitude positions in degrees corresponding to bathymetry
<i>ss</i> :	array of sidescan values in unknown units
<i>sslon</i> :	array of longitude positions in degrees corresponding to sidescan
<i>sslat</i> :	array of latitude positions in degrees corresponding to sidescan
<i>comment</i> :	comment string
<i>error</i> :	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about extract failures.

```
-----
int mb_insert(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int kind,
    int time_i[7],
    double time_d,
    double navlon,
    double navlat,
    double speed,
    double heading,
    int nbath,
    int namp,
    int nss,
    char *beamflag,
    double *bath,
    double *amp,
    double *bathacrosstrack,
    double *bathalongtrack,
```

```

double *ss,
double *ssacrosstrack,
double *ssalongtrack,
char *comment,
int *error);

```

The function **mb_insert** inserts sonar data into the structure pointed to by **store_ptr* according to the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function. The form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**; the *kind* value indicates which type of record is to be stored in **store_ptr*. Data will be inserted only if the data record is survey data (navigation, bathymetry, amplitude, and sidescan), navigation data (navigation only), or comment data (comment only). The values to be inserted are:

<i>kind:</i>	kind of data record inserted
	1 survey data
	2 comment
	12 navigation
<i>time_i:</i>	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d:</i>	time of current ping in seconds
	since 1/1/70 00:00:00
<i>navlon:</i>	longitude
<i>navlat:</i>	latitude
<i>speed:</i>	ship speed in km/s
<i>heading:</i>	ship heading in degrees
<i>distance:</i>	distance along shiptrack since last
	ping in km
<i>altitude:</i>	altitude of sonar above seafloor
	in m
<i>sonardepth:</i>	depth of sonar in m
<i>nbath:</i>	number of bathymetry values
<i>namp:</i>	number of amplitude values
<i>nss:</i>	number of sidescan values
<i>beamflag:</i>	array of bathymetry flags
<i>bath:</i>	array of bathymetry values in meters
<i>amp:</i>	array of amplitude values in unknown units
<i>bathacrosstrack:</i>	array of acrosstrack distances
	in meters corresponding to bathymetry
<i>bathalongtrack:</i>	array of alongtrack distances
	in meters corresponding to bathymetry
<i>ss:</i>	array of sidescan values in unknown units
<i>ssacrosstrack:</i>	array of acrosstrack distances
	in meters corresponding to sidescan
<i>ssalongtrack:</i>	array of alongtrack distances
	in meters corresponding to sidescan
<i>comment:</i>	comment string

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about insert failures.

```
-----
int mb_extract_nav(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int time_i[7],
    double *time_d,
    double *navlon,
    double *navlat,
    double *speed,
    double *heading,
    double *draft,
    double *roll,
    double *pitch,
    double *heave,
    int *error);
```

The function **mb_extract_nav** extracts navigation data from the structure pointed to by **store_ptr* according to the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function. The form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**; the *kind* value indicates which type of record is stored in **store_ptr*. Navigation data is returned if the data record is survey data (navigation, bathymetry, amplitude, and sidescan) or navigation data (navigation only).

The return values are:

<i>kind</i> :	kind of data record read
	1 survey data
	12 navigation
<i>time_i</i> :	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d</i> :	time of current ping in seconds
	since 1/1/70 00:00:00
<i>navlon</i> :	longitude
<i>navlat</i> :	latitude
<i>speed</i> :	ship speed in km/s
<i>heading</i> :	ship heading in degrees
<i>draft</i> :	sonar depth in meters
<i>roll</i> :	sonar roll in degrees
<i>pitch</i> :	sonar pitch in degrees
<i>heave</i> :	sonar heave in meters

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about extract failures.

```
-----
int mb_extract_nnav(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int nmax,
    int *kind,
    int *n,
    int *time_i,
    double *time_d,
    double *navlon,
    double *navlat,
    double *speed,
    double *heading,
    double *draft,
    double *roll,
    double *pitch,
    double *heave,
    int *error);
```

The function **mb_extract_nnav** extracts navigation in the same manner as **mb_extract_nav**, except that it returns as many as *nmax* navigation fixes associated with the current data record (some data formats, and the internally buffered asynchronous navigation used by **mb_navint_add** and related functions, can associate more than one navigation fix with a single survey ping). The number of fixes actually returned is placed in **n*, and the arrays *time_i*, *time_d*, *navlon*, *navlat*, *speed*, *heading*, *draft*, *roll*, *pitch*, and *heave* must each be allocated by the calling program to hold at least *nmax* values.

```
-----
int mb_insert_nav(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int time_i[7],
    double time_d,
    double navlon,
    double navlat,
    double speed,
    double heading,
    double draft,
    double roll,
    double pitch,
    double heave,
    int *error);
```

The function **mb_insert_nav** inserts navigation data into the structure pointed to by **store_ptr* according to the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function. The form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**; the *kind* value indicates which type of record is to be stored in **store_ptr*. Data will be inserted only if the data record is survey data (navigation, bathymetry, amplitude, and sidescan), or navigation data (navigation only). The values

to be inserted are:

<i>time_i</i> :	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d</i> :	time of current ping in seconds
	since 1/1/70 00:00:00
<i>navlon</i> :	longitude
<i>navlat</i> :	latitude
<i>speed</i> :	ship speed in km/s
<i>heading</i> :	ship heading in degrees
<i>draft</i> :	sonar depth in meters
<i>roll</i> :	sonar roll in degrees
<i>pitch</i> :	sonar pitch in degrees
<i>heave</i> :	sonar heave in meters

The return values are:

<i>error</i> :	error value
----------------	-------------

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about insert failures.

```
int mb_extract_altitude(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    double *transducer_depth,
    double *altitude,
    int *error);
```

The function **mb_extract_altitude** extracts the sonar transducer depth (**transducer_depth**) below the sea surface and the the sonar transducer **altitude** above the seafloor according to the **MBIO** descriptor pointed to by *mbio_ptr*. This function is not defined for all data formats. The *verbose* value controls the standard error output verbosity of the function. The form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**; the *kind* value indicates which type of record is stored in *store_ptr*. These data are returned only if the data record is survey data. These values are useful for sidescan processing applications. Both transducer depths and altitudes are reported in meters.

The return values are:

<i>kind</i> :	kind of data record read (error if not survey data):
	1 survey data
<i>transducer_depth</i> :	depth of sonar in meters
<i>altitude</i> :	altitude of sonar above seafloor in meters.
<i>error</i> :	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about data extraction failures.

```
-----
int mb_insert_altitude(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    double transducer_depth,
    double altitude,
    int *error);
```

The function **mb_insert_altitude** inserts sonar depth and altitude data into the structure pointed to by **store_ptr* according to the **MBIO** descriptor pointed to by *mbio_ptr*. This function is not defined for all data formats. The *verbose* value controls the standard error output verbosity of the function. The form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**. Data will be inserted only if the data record is survey data (navigation, bathymetry, amplitude, and sidescan). The values to be inserted are:

transducer_depth: depth of sonar in meters
altitude: altitude of sonar in meters

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about insert failures.

```
-----
int mb_extract_svp(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int *nsvp,
    double *depth,
    double *velocity,
    int *error);
```

The function **mb_extract_svp** extracts a water sound velocity profile according to the **MBIO** descriptor pointed to by *mbio_ptr*. This function is not defined for all data formats. The *verbose* value controls the standard error output verbosity of the function. The form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**; the *kind* value indicates which type of record is stored in **store_ptr*. These data are returned only if the data record is a sound velocity profile record. These values are useful for calculating bathymetry from travel times and beam angles.

The return values are:

kind: kind of data record read (error
if not SVP data):
6 SVP data
nsvp: number of depth and sound speed
data in the profile
depth: array of depths in meters

velocity: array of sound speeds in m/sec
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about data extraction failures.

```
int mb_insert_svp(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int nsvp,
    double *depth,
    double *velocity,
    int *error);
```

The function **mb_insert_svp** inserts a water sound velocity profile according to the **MBIO** descriptor pointed to by *mbio_ptr*. This function is not defined for all data formats. The *verbose* value controls the standard error output verbosity of the function. The form of the data structure is determined by the sonar system associated with the format of the data being read. A number of different data record types are recognized by **MB-System**. These data are inserted only if the data record is a sound velocity profile record. These values are useful for calculating bathymetry from travel times and beam angles. The inserted values are:

nsvp: number of depth and sound speed
 data in the profile
depth: array of depths in meters
velocity: array of sound speeds in m/sec

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about data insertion failures.

```
int mb_ttimes(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int *nbeams,
    double *ttimes,
    double *angles,
    double *angles_forward,
    double *angles_null,
    double *heave,
    double *alongtrack_offset,
    double *draft,
    double *ssv,
    int *error);
```

The function **mb_ttimes** extracts travel times and beam angles from a sonar-specific data structure pointed to by *store_ptr*. These values are used for calculating swath bathymetry. The *verbose* value controls the standard error output verbosity of the function. The coordinates of the beam angles can be a bit confusing. The angles

are returned in "takeoff angle coordinates" appropriate for raytracing. The array **angles** contains the angle from vertical and the array **angles_forward** contains the angle from across-track. This coordinate system is distinct from the roll-pitch coordinates appropriate for correcting roll and pitch values. A description of these relevant coordinate systems is given below. The **angles_null** array contains the effective sonar array orientation for each beam. The **angles_null** array may be used to correct beam angles using Snell's law if the **ssv** is changed. The **angles_null** values reflect the sonar configuration. For example, some multibeam sonars have a flat transducer array, and so the **angles_null** array consists of **nbeams** zero values. Other multibeams have circular arrays so that the **angles_null** values equal the **angles** values. The **alongtrack_offset** array accommodates sonars which report multiple pings in a single survey record; each ping occurs at a different position along the ship-track, producing along-track offsets relative to the navigation for some beam values. The sum of the **draft** value and the **heave** array values gives the depth of the sonar for each beam. For hull mounted installations the **draft** value is generally static but the **heave** values vary with time. For towed sonars the **draft** varies with time and the **heave** values are typically zero. The **ssv** value gives the water sound velocity at the sonar array.

The return values are:

kind:	kind of data record read (error if not survey data): 1 survey data
nbeams:	number of beams
ttimes:	array of two-way travel times in seconds
angles:	array of angles from vertical in degrees
angles_forward:	array of angles from across-track in degrees
angles_null:	array of sonar array orientation in degrees
heave:	array of heave values for each beam in meters
alongtrack_offset:	array of along-track distance offsets for each beam in meters
draft:	draft of sonar in meters
ssv:	water sound velocity at sonar in m/seconds
error:	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about data extraction failures.

```
int mb_detects(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int *nbeams,
    int *detects,
    int *error);
```

The function **mb_detects** extracts beam bottom detect types from a sonar-specific data structure pointed to by *store_ptr*. These values indicate whether the depth value associated with a particular beam *i* derived from an amplitude detect (e.g. *detects*[*i*] = 1), a phase detect (e.g. *detects*[*i*] = 2), or the algorithm is unknown (e.g. *detects*[*i*] = 0). The *verbose* value controls the standard error output verbosity of the function.

The return values are:

kind:	kind of data record read (error if not survey data): 1 survey data
--------------	--

<i>nbeams:</i>	number of beams	
<i>detects:</i>	array of <i>nbeams</i> bottom detect algorithm flags	
	0 = unknown	1 =
amplitude detect		2 = phase detect
<i>error:</i>	error value	

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about data extraction failures. This functionality is available for only a subset of the supported sonars. If the corresponding low level routine is undefined, **error* will be set to MB_ERROR_BAD_SYSTEM (14).

```
-----
int mb_gains(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    double *transmit_gain,
    double *pulse_length,
    double *receive_gain,
    int *error);
```

The function **mb_gains** extracts the most basic gain settings from a sonar-specific data structure pointed to by *store_ptr*. In many cases, sonars have more complicated gain functions, particularly with respect to the receiver TVG function. In those cases, the receive gain returned here refers to the constant gain setting and does not include any TVG parameters. The *verbose* value controls the standard error output verbosity of the function.

The return values are:

<i>kind:</i>	kind of data record read (error if not survey data):
	1 survey data
<i>transmit_gain:</i>	transmit gain (dB)
<i>pulse_length:</i>	transmit pulse length (sec)
<i>receive_gain:</i>	receive gain (dB)
<i>error:</i>	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about data extraction failures. This functionality is available for only a subset of the supported sonars. If the corresponding low level routine is undefined, **error* will be set to MB_ERROR_BAD_SYSTEM (14).

```
-----
int mb_pulses(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int *nbeams,
    int *pulses,
    int *error);
```

The function **mb_pulses** extracts, for each of the **nbeams* beams of the current data record, a code indicating

the type of transmit pulse (e.g. CW or FM chirp) used to obtain that beam, where the data format and sonar system record this information.

```
-----
int mb_sonarsettings(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    double *frequency,
    double *sample_rate,
    double *tx_pulse_width,
    double *power_selection,
    double *gain_selection,
    double *absorption,
    double *spreading,
    double *sound_velocity,
    double *beamwidth_tx,
    double *beamwidth_rx,
    int *error);
```

The function **mb_sonarsettings** extracts the sonar transmit and receive settings (operating frequency, sample rate, transmit pulse width, power and gain selections, assumed absorption and spreading coefficients, assumed sound velocity, and transmit and receive beam widths) in effect for the current data record. This functionality is available for only a subset of the supported sonars; if the corresponding low level routine is undefined, **error* will be set to MB_ERROR_BAD_SYSTEM (14).

```
-----
int mb_makess(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int pixel_size_set,
    double *pixel_size,
    int swath_width_set,
    double *swath_width,
    int pixel_int,
    int *error);
```

The function **mb_makess** constructs a processed sidescan swath (uniformly spaced in across-track distance) from the raw sidescan and/or bathymetry of the current data record. If *pixel_size_set* is nonzero, **pixel_size* is used as supplied; otherwise a pixel size is calculated automatically and returned in **pixel_size*. Likewise, if *swath_width_set* is nonzero **swath_width* is used as supplied, and otherwise a swath width is calculated and returned. If *pixel_int* is nonzero, pixels with no assigned sidescan value are interpolated from neighboring pixels. The resulting processed sidescan values are stored back into the sonar-specific structure pointed to by *store_ptr* for later extraction by **mb_extract** or **mb_get_all**.

```
-----
int mb_extract_rawssdimensions(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
```

```
double *sample_interval,
int *num_samples_port,
int *num_samples_stbd,
int *error);
```

The function **mb_extract_rawssdimensions** returns the sample interval and the numbers of port and starboard side raw sidescan samples available in the current data record, without extracting the raw sidescan sample values themselves. This allows a program to size arrays appropriately before calling **mb_extract_rawss**.

```
int mb_extract_rawss(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int *sidescan_type,
    double *sample_interval,
    double *beamwidth_xtrack,
    double *beamwidth_ltrack,
    int *num_samples_port,
    double *rawss_port,
    int *num_samples_stbd,
    double *rawss_stbd,
    int *error);
```

The function **mb_extract_rawss** extracts the "raw" (as opposed to slant-range-corrected or otherwise processed) sidescan sample arrays for the port and starboard sides of the current data record, where the data format supports this. Since the meaning of "raw" sidescan and the way it is stored varies greatly among sonars, the values of **sidescan_type*, **sample_interval*, **beamwidth_xtrack*, and **beamwidth_ltrack* allow the calling program to interpret the returned sample arrays **rawss_port* and **rawss_stbd* (of length **num_samples_port* and **num_samples_stbd* respectively) correctly. This function is not implemented for all data formats.

```
int mb_insert_rawss(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int kind,
    int sidescan_type,
    double sample_interval,
    double beamwidth_xtrack,
    double beamwidth_ltrack,
    int num_samples_port,
    double *rawss_port,
    int num_samples_stbd,
    double *rawss_stbd,
    int *error);
```

The function **mb_insert_rawss** inserts "raw" port and starboard sidescan sample arrays into the data record pointed to by *store_ptr*, where the data format supports this. The parameters have the same meaning as the corresponding return values of **mb_extract_rawss**. This function is not implemented for all data formats.

```

-----
int mb_extract_segytraceheader(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    void *segytraceheader_ptr,
    int *error);

```

The function **mb_extract_segytraceheader** extracts a **SEG Y** trace header (a *struct mb_segytraceheader_struct*, supplied by the calling program and filled in by this function) describing the current data record, for data formats that carry subbottom profiler or other seismic reflection data that can be represented in **SEG Y** form (see the **mbsegylist** and **mbsegygrid** manual pages).

```

-----
int mb_extract_segy(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *sampleformat,
    int *kind,
    void *segytraceheader_ptr,
    float *segydata,
    int *error);

```

The function **mb_extract_segy** extracts both the **SEG Y** trace header (as does **mb_extract_segytraceheader**) and the associated trace sample data for the current data record, placing the samples in the array **segydata* (allocated by the calling program to hold at least *nsamp* values as specified in the trace header). The value **sampleformat* indicates how the trace samples are to be interpreted (e.g. **MB_SEGY_SAMPLEFORMAT_TRACE**, **MB_SEGY_SAMPLEFORMAT_ENVELOPE**, or **MB_SEGY_SAMPLEFORMAT_ANALYTIC**).

```

-----
int mb_insert_segy(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int kind,
    void *segytraceheader_ptr,
    float *segydata,
    int *error);

```

The function **mb_insert_segy** inserts a **SEG Y** trace header and its associated trace sample data into the data record pointed to by *store_ptr*, where the data format supports this. The parameters have the same meaning as the corresponding arguments of **mb_extract_segy**.

```

-----
int mb_ctd(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int *nctd,

```



```

double *time_d,
double *conductivity,
double *temperature,
double *depth,
double *salinity,
double *soundspeed,
int *error);

```

The function **mb_ctd** extracts a set of **nctd* CTD (conductivity-temperature-depth) measurements associated with the current data record, where the data format carries CTD data (e.g. some AUV data formats). The arrays *time_d*, *conductivity*, *temperature*, *depth*, *salinity*, and *soundspeed* must each be allocated by the calling program to hold at least **MB_CTD_MAX** values.

```

-----
int mb_ancilliarysensor(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    int *kind,
    int *nsensor,
    double *time_d,
    double *sensor1,
    double *sensor2,
    double *sensor3,
    double *sensor4,
    double *sensor5,
    double *sensor6,
    double *sensor7,
    double *sensor8,
    int *error);

```

The function **mb_ancilliarysensor** extracts a set of **nsensor* time-stamped samples from up to eight generic auxiliary sensor channels associated with the current data record, where the data format carries such data (e.g. some AUV data formats). The arrays *time_d*, *sensor1* through *sensor8* must each be allocated by the calling program to hold at least **MB_CTD_MAX** values.

```

-----
int mb_copyrecord(
    int verbose,
    void *mbio_ptr,
    void *store_ptr,
    void *copy_ptr,
    int *error);

```

The function **mb_copyrecord** copies the sonar-specific data structure pointed to by *store_ptr* into the data structure pointed to by **copy_ptr*. The data structures must already have been allocated.

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about data copy failures.

```
-----
int mb_indextable(
    int verbose,
    void *mbio_ptr,
    int *num_indextable,
    void **indextable_ptr,
    int *error);
```

The function **mb_indextable** returns the number of entries **num_indextable* and a pointer **indextable_ptr* to an index table built by **MBIO** while reading the file associated with *mbio_ptr*. Each entry of the index table records the byte offset, kind, and time of a data record already read, allowing an application (e.g. **mbindex** or **mbnavadjust**) to later seek directly to particular records without a further sequential read.

```
-----
int mb_indextablefix(
    int verbose,
    void *mbio_ptr,
    int num_indextable,
    void *indextable_ptr,
    int *error);
```

The function **mb_indextablefix** passes an index table of *num_indextable* entries (typically obtained from **mb_indextable** and possibly edited by the calling program) to the format-specific fixing routine registered for the file associated with *mbio_ptr*, where the data format supports this. This is used to make in-place corrections to records identified by the index table.

```
-----
int mb_indextableapply(
    int verbose,
    void *mbio_ptr,
    int num_indextable,
    void *indextable_ptr,
    int n_file,
    int *error);
```

The function **mb_indextableapply** applies an index table of *num_indextable* entries to the *n_file*th parallel data file (see **mb_format_info**) associated with *mbio_ptr*, using the format-specific apply routine registered for that format, where the data format supports this.

LEVEL 3 FUNCTIONS

```
int mb_buffer_init(
    int verbose,
    void **buff_ptr,
    int *error);
```

The function **mb_buffer_init** initializes the data structures required for buffered i/o. A pointer to the buffer data structure is returned as **buff_ptr*. The *verbose* value controls the standard error output verbosity of the function.

The return values are:

<i>*buff_ptr</i> :	pointer to buffer structure
<i>error</i> :	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about buffer initialization failures.

```
-----
int mb_buffer_close(
    int verbose,
    void **buff_ptr,
    void *mbio_ptr,
    int *error);
```

The function **mb_buffer_close** releases all memory allocated for buffered i/o, including the structure pointed to by **buff_ptr*. The *verbose* value controls the standard error output verbosity of the function.

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about buffer deallocation failures.

```
-----
int mb_buffer_load(
    int verbose,
    void *buff_ptr,
    void *mbio_ptr,
    int nwant,
    int *nload,
    int *nbuff,
    int *error);
```

The function **mb_buffer_load** loads data into the buffer pointed to by *buff_ptr* from the input file initialized in the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function.

The input control parameters have the following significance:

nwant: The number of data records desired
in the buffer.

The returned values are:

nload: The number of data records loaded into the buffer.
nbuff: The total number of data records in the buffer after loading.
error: error value

The buffer may already contain data records when the **mb_buffer_load** call is made; if the number of previously loaded records is less than *nwant*, the function will attempt to read and load records until a total of *nwant* records are loaded. The *nload* value is the number of data records loaded during the current function call, and the *nbuff* value is the number of data records in the buffer at the completion of the **mb_buffer_load** call. A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about buffer deallocation failures.

```
-----
int mb_buffer_dump(
    int verbose,
    void *buff_ptr,
```

```

void *mbio_ptr,
void *ombio_ptr,
int nhold,
int *ndump,
int *nbuff,
int *error);

```

The function **mb_buffer_dump** dumps data from the buffer pointed to by **buff_ptr* into the output file initialized in the **MBIO** descriptor pointed to by *ombio_ptr*. The data in the buffer were read from the input file initialized in the **MBIO** descriptor pointed to by *mbio_ptr*. The *verbose* value controls the standard error output verbosity of the function.

The input control parameters have the following significance:

nhold: The number of data records desired to be held
 in the buffer.

The returned values are:

nload: The number of data records dumped from the buffer.
nbuff: The total number of data records in the buffer after dumping.
error: error value

If the number of loaded records is more than *nhold*, the function will attempt to write out records from the beginning of the buffer until *nhold* records are left in the buffer. The *ndump* value is the number of data records dumped during the current function call, and the *nbuff* value is the number of data records in the buffer at the completion of the **mb_buffer_dump** call. A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about buffer deallocation failures.

```

int mb_buffer_clear(
    int verbose,
    void *buff_ptr,
    void *mbio_ptr,
    int nhold,
    int *ndump,
    int *nbuff,
    int *error);

```

The function **mb_buffer_clear** removes data from the buffer pointed to by **buff_ptr* without writing those data records to an output file. An **MBIO** descriptor pointed to by *mbio_ptr* is still required, and generally represents the **MBIO** descriptor used to read and load the data originally. The *verbose* value controls the standard error output verbosity of the function.

The input control parameters have the following significance:

nhold: The number of data records desired to be held
 in the buffer.

The returned values are:

ndump: The number of data records cleared from the buffer.
nbuff: The total number of data records in the buffer after dumping.
error: error value

If the number of loaded records is more than *nhold*, the function will attempt to clear out records from the beginning of the buffer until *nhold* records are left in the buffer. The *ndump* value is the number of data records

cleared during the current function call, and the *nbuff* value is the number of data records in the buffer at the completion of the **mb_buffer_dump** call. A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about buffer deallocation failures.

```
int mb_buffer_get_kind(
    int verbose,
    void *buff_ptr,
    void *mbio_ptr,
    int id,
    int *kind,
    int *error);
```

The function **mb_buffer_get_kind** returns the data record kind **kind* (e.g. survey data, comment, navigation) of the buffer entry numbered *id* in the buffer pointed to by **buff_ptr*. An **MBIO** descriptor pointed to by *mbio_ptr* is still required, and generally represents the **MBIO** descriptor used to read and load the data originally. The *verbose* value controls the standard error output verbosity of the function.

The input control parameters have the following significance:

id: id number of the data record of interest

The returned values are:

kind: kind of data record found
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_buffer_get_next_data(
    int verbose,
    void *buff_ptr,
    void *mbio_ptr,
    int start,
    int *id,
    int time_i[7],
    double *time_d,
    double *navlon,
    double *navlat,
    double *speed,
    double *heading,
    int *nbath,
    int *namp,
    int *nss,
    char *beamflag,
    double *bath,
    double *amp,
    double *bathacrosstrack,
    double *bathalongtrack,
    double *ss,
    double *ssacrosstrack,
    double *ssalongtrack,
    int *error);
```

The function **mb_buffer_get_next_data** searches for the next survey data record in the buffer, beginning at buffer index *start*. Since buffer indexes begin at 0, the first call to **mb_buffer_get_next_data** should have *start* = 0. If a survey data record is found at or beyond *start*, **mb_buffer_get_next_data** returns the buffer index of that record in *id*. Data is also returned in the forms of bathymetry, amplitude, and sidescan survey data. No comments or other non-survey data records are returned. The *verbose* value controls the standard error output verbosity of the function.

The input control parameters have the following significance:

start: The buffer index at which to start searching for a survey data record.

The returned values are:

<i>id</i> :	The buffer index of the first survey data record at or after <i>start</i> .		
<i>time_i</i> :	time of current ping		
	<i>time_i</i> [0]: year		
	<i>time_i</i> [1]: month		
	<i>time_i</i> [2]: day		
	<i>time_i</i> [3]: hour		
	<i>time_i</i> [4]: minute		
	<i>time_i</i> [5]: second		
	<i>time_i</i> [6]: microsecond		
<i>time_d</i> :	time of current ping in seconds since 1/1/70 00:00:00		
<i>navlon</i> :	longitude		
<i>navlat</i> :	latitude		
<i>speed</i> :	ship speed in km/s		
<i>heading</i> :	ship heading in degrees		
<i>nbath</i> :	number of bathymetry values		
<i>namp</i> :	number of amplitude values		
<i>nss</i> :	number of sidescan values	<i>beamflag</i> :	array of bathymetry
flags			
<i>bath</i> :	array of bathymetry values in meters		
<i>amp</i> :	array of amplitude values in unknown units		
<i>bathacrosstrack</i> :	array of acrosstrack distances in meters corresponding to bathymetry		
<i>bathalongtrack</i> :	array of alongtrack distances in meters corresponding to bathymetry		
<i>ss</i> :	array of sidescan values in unknown units		
<i>ssacrosstrack</i> :	array of acrosstrack distances in meters corresponding to sidescan		
<i>ssacrosstrack</i> :	array of alongtrack distances in meters corresponding to sidescan		
<i>error</i> :	error value		

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures. The most common error occurs when no more survey data records remain to be found in the buffer; in this case, *error* = -14.

```
int mb_buffer_extract(
    int verbose,
    void *buff_ptr,
```

```

void *mbio_ptr,
int id,
int *kind,
int time_i[7],
double *time_d,
double *navlon,
double *navlat,
double *speed,
double *heading,
int *nbath,
int *namp,
int *nss,
char *beamflag,
double *bath,
double *amp,
double *bathacrosstrack,
double *bathalongtrack,
double *ss,
double *ssacrosstrack,
double *ssalongtrack,
char *comment,
int *error);

```

The function **mb_buffer_extract** extracts and returns a subset of the data in a buffer record. The *verbose* value controls the standard error output verbosity of the function. The buffer record is specified with the buffer index *id*. The data is either in the form of bathymetry, amplitude, and sidescan survey data or a comment string.

The input control parameters have the following significance:

id: The buffer index of the data record to extract.

The returned values are:

<i>kind</i> :	kind of data record extracted
	1 survey data
	2 comment
	>=3 other data that cannot be passed by mb_buffer_extract
<i>time_i</i> :	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d</i> :	time of current ping in seconds since 1/1/70 00:00:00
<i>navlon</i> :	longitude
<i>navlat</i> :	latitude
<i>speed</i> :	ship speed in km/s
<i>heading</i> :	ship heading in degrees
<i>nbath</i> :	number of bathymetry values

<i>namp:</i>	number of amplitude values
<i>nss:</i>	number of sidescan values
<i>beamflag:</i>	array of bathymetry flags
<i>bath:</i>	array of bathymetry values in meters
<i>amp:</i>	array of amplitude values in unknown units
<i>bathacrosstrack:</i>	array of acrosstrack distances in meters corresponding to bathymetry
<i>bathalongtrack:</i>	array of alongtrack distances in meters corresponding to bathymetry
<i>ss:</i>	array of sidescan values in unknown units
<i>ssacrosstrack:</i>	array of acrosstrack distances in meters corresponding to sidescan
<i>ssalongtrack:</i>	array of alongtrack distances in meters corresponding to sidescan
<i>comment:</i>	comment string
<i>error:</i>	error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about extract failures.

```
int mb_buffer_insert(
    int verbose,
    void *buff_ptr,
    void *mbio_ptr,
    int id,
    int time_i[7],
    double time_d,
    double navlon,
    double navlat,
    double speed,
    double heading,
    int nbath,
    int namp,
    int nss,
    char *beamflag,
    double *bath,
    double *amp,
    double *bathacrosstrack,
    double *bathalongtrack,
    double *ss,
    double *ssacrosstrack,
    double *ssalongtrack,
    char *comment,
    int *error);
```

The function **mb_buffer_insert** inserts data into a buffer record, replacing a subset of the original values. The *verbose* value controls the standard error output verbosity of the function. The buffer record is specified with the buffer index *id*. The data is either in the form of bathymetry, amplitude, and sidescan survey data or a comment string.

The input control parameters have the following significance:

<i>id:</i>	The buffer index of the data
------------	------------------------------

record to insert.

The returned values are:

<i>kind:</i>	kind of data record inserted
	1 survey data
	2 comment
	>=3 other data that cannot be passed by mb_buffer_insert
<i>time_i:</i>	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d:</i>	time of current ping in seconds since 1/1/70 00:00:00
<i>navlon:</i>	longitude
<i>navlat:</i>	latitude
<i>speed:</i>	ship speed in km/s
<i>heading:</i>	ship heading in degrees
<i>nbath:</i>	number of bathymetry values
<i>namp:</i>	number of amplitude values
<i>nss:</i>	number of sidescan values
<i>beamflag:</i>	array of bathymetry flags
<i>bath:</i>	array of bathymetry values in meters
<i>amp:</i>	array of amplitude values in unknown units
<i>bathacrosstrack:</i>	array of acrosstrack distances in meters corresponding to bathymetry
<i>bathalongtrack:</i>	array of alongtrack distances in meters corresponding to bathymetry
<i>ss:</i>	array of sidescan values in unknown units
<i>ssacrosstrack:</i>	array of acrosstrack distances in meters corresponding to sidescan
<i>ssalongtrack:</i>	array of alongtrack distances in meters corresponding to sidescan
<i>comment:</i>	comment string

The returned values are:

<i>error:</i>	error value
---------------	-------------

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about insert failures.

```
-----
int mb_buffer_get_next_nav(
    int verbose,
    void *buff_ptr,
    void *mbio_ptr,
    int start,
    int *id,
    int time_i[7],
```

```

double *time_d,
double *navlon,
double *navlat,
double *speed,
double *heading,
double *draft,
double *roll,
double *pitch,
double *heave,
int *error);

```

The function **mb_buffer_get_next_nav** searches for the next survey data record in the buffer, beginning at buffer index *start*. Since buffer indexes begin at 0, the first call to **mb_buffer_get_next_nav** should have *start* = 0. If a survey data record is found at or beyond *start*, **mb_buffer_get_next_nav** returns the buffer index of that record in *id*. Navigation and vertical reference sensor data is also returned. No comments or other non-survey data records are returned. The *verbose* value controls the standard error output verbosity of the function.

The input control parameters have the following significance:

start: The buffer index at which to start searching for a survey data record.

The returned values are:

<i>id</i> :	The buffer index of the first survey data record at or after <i>start</i> .
<i>time_i</i> :	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d</i> :	time of current ping in seconds
	since 1/1/70 00:00:00
<i>navlon</i> :	longitude
<i>navlat</i> :	latitude
<i>speed</i> :	ship speed in km/s
<i>heading</i> :	ship heading in degrees
<i>roll</i> :	ship roll in degrees
<i>pitch</i> :	ship pitch in degrees
<i>heave</i> :	ship heave in meters

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures. The most common error occurs when no more survey data records remain to be found in the buffer; in this case, *error* = -14.

```

int mb_buffer_extract_nav(
    int verbose,
    void *buff_ptr,
    void *mbio_ptr,
    int id,
    int *kind,

```

```

    int time_i[7],
    double *time_d,
    double *navlon,
    double *navlat,
    double *speed,
    double *heading,
    double *draft,
    double *roll,
    double *pitch,
    double *heave,
    int *error);

```

The function **mb_buffer_extract_nav** extracts and returns a subset of the data in a buffer record. The *verbose* value controls the standard error output verbosity of the function. The buffer record is specified with the buffer index *id*. The data returned consists of navigation and vertical reference sensor data.

The input control parameters have the following significance:

id: The buffer index of the data record to extract.

The returned values are:

<i>kind</i> :	kind of data record extracted
	1 survey data
	2 comment
	>=3 other data that cannot be passed by mb_buffer_extract_nav
<i>time_i</i> :	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d</i> :	time of current ping in seconds since 1/1/70 00:00:00
<i>navlon</i> :	longitude
<i>navlat</i> :	latitude
<i>speed</i> :	ship speed in km/s
<i>heading</i> :	ship heading in degrees
<i>roll</i> :	ship roll in degrees
<i>pitch</i> :	ship pitch in degrees
<i>heave</i> :	ship heave in meters

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about extract failures.

```

int mb_buffer_insert_nav(
    int verbose,
    void *buff_ptr,
    void *mbio_ptr,
    int id,

```

```

    int time_i[7],
    double time_d,
    double navlon,
    double navlat,
    double speed,
    double heading,
    double draft,
    double roll,
    double pitch,
    double heave,
    int *error);

```

The function **mb_buffer_insert_nav** inserts navigation and vertical reference sensor data into a buffer record, replacing a subset of the original values. The *verbose* value controls the standard error output verbosity of the function. The buffer record is specified with the buffer index *id*.

The input control parameters have the following significance:

id: The buffer index of the data record to insert.

The returned values are:

<i>kind</i> :	kind of data record inserted
	1 survey data
	2 comment
	>=3 other data that cannot be passed by mb_buffer_insert_nav
<i>time_i</i> :	time of current ping
	<i>time_i</i> [0]: year
	<i>time_i</i> [1]: month
	<i>time_i</i> [2]: day
	<i>time_i</i> [3]: hour
	<i>time_i</i> [4]: minute
	<i>time_i</i> [5]: second
	<i>time_i</i> [6]: microsecond
<i>time_d</i> :	time of current ping in seconds since 1/1/70 00:00:00
<i>navlon</i> :	longitude
<i>navlat</i> :	latitude
<i>speed</i> :	ship speed in km/s
<i>heading</i> :	ship heading in degrees
<i>roll</i> :	ship roll in degrees
<i>pitch</i> :	ship pitch in degrees
<i>heave</i> :	ship heave in meters

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about insert failures.

```

int mb_buffer_get_ptr(
    int verbose,
    void *buff_ptr,
    void *mbio_ptr,
    int id,

```

```
void **store_ptr,
int *error);
```

The function **mb_buffer_get_ptr** returns a pointer to the data structure in a buffer record. The *verbose* value controls the standard error output verbosity of the function. The buffer record is specified with the buffer index *id*. The data returned consists of a pointer to the data structure stored in the specified buffer record.

The input control parameters have the following significance:

id: The buffer index of the data
 record to locate.

The return values are:

**store_ptr*: pointer to data in specified
 buffer record
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about buffer failures.

MISCELLANEOUS FUNCTIONS

```
int mb_version(
    int verbose,
    char *version_string,
    int *version_id,
    int *version_major,
    int *version_minor,
    int *version_archive,
    int *error);
```

The function **mb_version** returns the **MB-System** release version as both a string **version_string* (e.g. "5.8.0") and, parsed from that string, the major, minor, and archive version numbers and a single combined numeric **version_id* (e.g. version 5.8.2303 becomes 50802303) suitable for numeric comparisons.

```
-----
int mb_user_host_date(
    int verbose,
    char user[256],
    char host[256],
    char date[32],
    int *error);
```

The function **mb_user_host_date** returns the current user name *user* (from the **USER** or **LOGNAME** environment variable), host name *host*, and current date and time *date* as strings. This is used by many **MB-System** programs to record who processed a file, on what machine, and when, typically in a processing comment record.

```
-----
int mb_default_defaults(
    int verbose,
    int *format,
    int *pings,
    int *lonflip,
    double bounds[4],
```

```

    int *btime_i,
    int *etime_i,
    double *speedmin,
    double *timegap);

```

The function **mb_default_defaults** sets the hardwired **MBIO** default control parameter values in the same manner as **mb_defaults** below, but without consulting a possible *.mbio_defaults* file in the user's home directory.

```

int mb_defaults(
    int verbose,
    int *format,
    int *pings,
    int *lonflip,
    double bounds[4],
    int *btime_i,
    int *etime_i,
    double *speedmin,
    double *timegap);

```

The function **mb_defaults** provides default values of control parameters used by some of the **MBIO** functions. The *verbose* value controls the standard error output verbosity of the function. The other parameters are set by the function; the meaning of these parameters is discussed in the listings of the functions **mb_read_init** and **mb_write_init**. If an *.mbio_defaults* file exists in the user's home directory, the *lonflip* and *timegap* defaults are read from this file. Otherwise, the values are set as:

```

    *lonflip = 0
    *timegap = 1

```

The other values are simply set as:

```

    *format = 0
    *pings = 1
    bounds[0] = -360.
    bounds[1] = 360.
    bounds[2] = -90.
    bounds[3] = 90.
    btime_i[0] = 1962;
    btime_i[1] = 2;
    btime_i[2] = 21;
    btime_i[3] = 10;
    btime_i[4] = 30;
    btime_i[5] = 0;
    btime_i[6] = 0;
    etime_i[0] = 2062;
    etime_i[1] = 2;
    etime_i[2] = 21;
    etime_i[3] = 10;
    etime_i[4] = 30;
    etime_i[5] = 0;
    etime_i[6] = 0;
    *speedmin = 0.0

```

A status value is returned to indicate success or failure.

```
-----
int mb_lonflip(
    int verbose,
    int *lonflip);
```

The function **mb_lonflip** returns just the default **lonflip* value described above under **mb_defaults**, reading it from a *.mbio_defaults* file in the user's home directory if one exists (otherwise 0).

```
-----
int mb_fbtversion(
    int verbose,
    int *fbtversion);
```

The function **mb_fbtversion** returns the version number of the fast bathymetry (*.fbt*) file format to be generated by **mb_make_info** and related functions, reading it from a *.mbio_defaults* file in the user's home directory if one exists (otherwise 3, the current *.fbt* format version).

```
-----
int mb_uselockfiles(
    int verbose,
    bool *uselockfiles);
```

The function **mb_uselockfiles** returns true if **MBIO** should create lock files to prevent simultaneous editing of swath data files by multiple programs (e.g. **mbedit**), reading this setting from a *.mbio_defaults* file in the user's home directory if one exists (otherwise true). On Windows, **uselockfiles* is always returned false because lock file creation is not supported there.

```
-----
int mb_fileiobuffer(
    int verbose,
    int *fileiobuffer);
```

The function **mb_fileiobuffer** returns the size (in some implementation-defined unit) of the buffer to be used for low level file input/output by **mb_fileio_open**, reading this setting from a *.mbio_defaults* file in the user's home directory if one exists (otherwise 0, meaning the system default buffering is used).

```
-----
int mb_mbview_defaults(
    int verbose,
    int *primary_colortable,
    int *primary_colortable_mode,
    int *primary_shade_mode,
    int *slope_colortable,
    int *slope_colortable_mode,
    int *secondary_colortable,
    int *secondary_colortable_mode,
    double *illuminate_magnitude,
    double *illuminate_elevation,
    double *illuminate_azimuth,
    double *slope_magnitude);
```

The function **mb_mbview_defaults** returns default color table, shading mode, and illumination parameter values used by the **mbview** 3D visualization widget, reading them from a *.mbio_defaults* file in the user's home

directory if one exists.

```
-----
int mb_env(
    int verbose,
    char *psdisplay,
    char *imgdisplay,
    char *mbproject);
```

The function **mb_env** provides default values of Postscript and image display programs invoked by some **MB-System** programs and macros, and a default value for a working project name that will be used by future applications. The *verbose* value controls the standard error output verbosity of the function. If an *.mbio_defaults* file exists in the user's home directory, the **psdisplay*, **imgdisplay*, **mbproject* defaults are read from this file. Otherwise, the values are set as:

```
psdisplay = "xpsview" (IRIX OS)
             "pageview" (Solaris OS)
             "gv" (other OS)
             "ghostview" (other OS)
imgdisplay = "gimp" (Linux OS)
             "xv" (other than Linux OS)
mbproject = "none"
```

```
-----
int mb_format(
    int verbose,
    int *format,
    int *error);
```

Given the format identifier *format*, **mb_format** checks if the format is valid. If the format id corresponds to a value used in previous (<4.00) versions of **MB-System**, then the format value will be aliased to the current corresponding value.

The return values are:

```
format:      MBIO format id
error:      error value
```

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
-----
int mb_format_register(
    int verbose,
    int *format,
    void *mbio_ptr,
    int *error);
```

The function **mb_format_register** is called by **mb_read_init** and **mb_write_init** and serves to load format specific parameters and function parameters into the **MBIO** control structure pointed to by **error*. The format id **format* is first checked for validity. In some cases, formerly valid but now obsolete format id values are mapped to current values.

The input values are:

```
*format:      MBIO format id
```


**mbio_ptr*: pointer to data in specified
buffer record

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_format_info(
    int verbose,
    int *format,
    int *system,
    int *beams_bath_max,
    int *beams_amp_max,
    int *pixels_ss_max,
    char *format_name,
    char *system_name,
    char *format_description,
    int *numfile,
    int *filetype,
    bool *variable_beams,
    bool *traveltime,
    bool *beam_flagging,
    int *platform_source,
    int *nav_source,
    int *sensordepth_source,
    int *heading_source,
    int *attitude_source,
    int *svp_source,
    double *beamwidth_xtrack,
    double *beamwidth_ltrack,
    int *error);
```

The function **mb_format_info** returns a variety of data format specific parameters. The format id **format* is first checked for validity. In some cases, formerly valid but now obsolete format id values are mapped to current values.

The input values are:

format*: **MBIO format id

The return values are:

format*: **MBIO format id
system*: **MBIO sonar system id
**beams_bath_max*: maximum number of bathymetry beams
**beams_amp_max*: maximum number of amplitude beams
**pixels_ss_max*: maximum number of sidescan pixels
format_name*: **MBIO format name
system_name*: **MBIO sonar system name
format_description*: **MBIO format description
**numfile*: number of parallel data files used in format
**filetype*: type of data files

**variable_beams*: number of beams can vary [boolean]
**traveltime*: travel time data available [boolean]
**beam_flagging*: beam flagging supported [boolean]
**platform_source*: kind of data records containing
 sensor platform (mounting/offset)
 information
**nav_source*: kind of data records containing navigation
**sensordepth_source*: kind of data records containing
 sensor (transducer) depth
**heading_source*: kind of data records containing
 heading
**attitude_source*: kind of data records containing
 attitude (roll, pitch, heave)
**svp_source*: kind of data records containing
 sound velocity profiles
**beamwidth_xtrack*: typical athwartships beam
 width [degrees]
**beamwidth_ltrack*: typical alongtrack beam
 width [degrees]
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_format_system(
    int verbose,
    int *format,
    int *system,
    int *error);
```

The function **mb_format_system** returns the **MBIO** sonar system id. The format id **format* is first checked for validity. In some cases, formerly valid but now obsolete format id values are mapped to current values. The input values are:

format*: **MBIO format id

The return values are:

format*: **MBIO format id
system*: **MBIO sonar system id
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_format_description(
    int verbose,
    int *format,
    char *description,
    int *error);
```

The function **mb_format_description** returns a short description of the format in the string **description*. The format id **format* is first checked for validity. In some cases, formerly valid but now obsolete format id values are mapped to current values. The input values are:

format*: **MBIO format id

The return values are:

format*: **MBIO format id
format_description*: **MBIO format description
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_format_dimensions(
    int verbose,
    int *format,
    int *beams_bath_max,
    int *beams_amp_max,
    int *pixels_ss_max,
    int *error);
```

The function **mb_format_dimensions** returns the maximum numbers of beams and pixels associated with a particular data format. The format id **format* is first checked for validity. In some cases, formerly valid but now obsolete format id values are mapped to current values. The input values are:

format*: **MBIO format id

The return values are:

format*: **MBIO format id
**beams_bath_max*: maximum number of bathymetry beams
**beams_amp_max*: maximum number of amplitude beams
**pixels_ss_max*: maximum number of sidescan pixels
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_format_flags(
    int verbose,
    int *format,
    bool *variable_beams,
    bool *traveltime,
    bool *beam_flagging,
    int *error);
```

The function **mb_format_flags** returns flags indicating certain characteristics of the specified data format. The format id **format* is first checked for validity. In some cases, formerly valid but now obsolete format id values are mapped to current values. The input values are:

format*: **MBIO format id

The return values are:

format*: **MBIO format id
**variable_beams*: number of beams can vary [boolean]
**traveltime*: travel time data available [boolean]
**beam_flagging*: beam flagging supported [boolean]
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_format_source(
```

```

    int verbose,
    int *format,
    int *platform_source,
    int *nav_source,
    int *sensordepth_source,
    int *heading_source,
    int *attitude_source,
    int *svp_source,
    int *error);

```

The function **mb_format_source** returns flags indicating what kinds of data records contain platform, navigation, sensor depth, heading, attitude, and sound velocity profile values in the specified data format. The format id **format* is first checked for validity. In some cases, formerly valid but now obsolete format id values are mapped to current values. The input values are:

format*: **MBIO format id

The return values are:

```

*format:           MBIO format id
*platform_source:     kind of data records containing
                       sensor platform information
*nav_source:        kind of data records containing
                       navigation
*sensordepth_source:   kind of data records containing
                       sensor (transducer) depth
*heading_source:    kind of data records containing
                       heading
*attitude_source:    kind of data records containing
                       attitude
*svp_source:        kind of data records containing
                       sound velocity profiles
error:              error value

```

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```

int mb_format_beamwidth(
    int verbose,
    int *format,
    double *beamwidth_xtrack,
    double *beamwidth_ltrack,
    int *error);

```

The function **mb_format_beamwidth** returns typical, upper bound values for athwartships and alongtrack beam widths. The format id **format* is first checked for validity. In some cases, formerly valid but now obsolete format id values are mapped to current values. The input values are:

format*: **MBIO format id

The return values are:

```

*format:           MBIO format id
*beamwidth_xtrack:   typical athwartships beam
                       width [degrees]
*beamwidth_ltrack:   typical alongtrack beam
                       width [degrees]
error:              error value

```

A status value indicating success or failure is returned; the error value argument *error* passes more detailed

information about failures.

```
-----
int mb_get_format(
    int verbose,
    char *filename,
    char *fileroot,
    int *format,
    int *error);
```

The function **mb_get_format** attempts to determine the **MBIO** format id **format* of *filename* from a recognized filename suffix convention (e.g. *.mbXXX*, *.mb1*, or *.nve*), and if successful returns in **fileroot* the part of *filename* preceding that suffix. If no recognized suffix is found, **format* is returned as 0.

```
-----
int mb_get_relative_path(
    int verbose,
    char *path,
    char *pwd,
    int *error);
```

The function **mb_get_relative_path** rewrites **path* in place, replacing it with the equivalent path relative to the working directory *pwd*, if *path* lies within (or below) *pwd*. This is used, for instance, by **mbdatalist** so that datalist files can reference data files with relative rather than absolute paths.

```
-----
int mb_get_absolute_path(
    int verbose,
    char *path,
    char *pwd,
    int *error);
```

The function **mb_get_absolute_path** rewrites **path* in place, replacing a relative path with the equivalent absolute path obtained by prepending the working directory *pwd*, if *path* is not already absolute.

```
-----
int mb_get_shortest_path(
    int verbose,
    char *path,
    int *error);
```

The function **mb_get_shortest_path** rewrites **path* in place with the shortest equivalent path, removing redundant *./* and *../* path components.

```
-----
int mb_get_basename(
    int verbose,
    char *path,
    int *error);
```

The function **mb_get_basename** rewrites **path* in place, replacing it with just its final path component (the part following the last */*).

```
int mb_cvt_to_nix_path(
    char *path);
```

The function **mb_cvt_to_nix_path**, available only when **MB-System** is built for Windows, rewrites *path* in place, converting a Windows-style path (using "\e" separators and a drive letter) into the equivalent Unix-style path (using "/" separators, with any drive letter removed) used internally by **MBIO**.

```
int mb_datalist_open(
    int verbose,
    void **datalist_ptr,
    char *path,
    int look_processed,
    int *error);
```

The function **mb_datalist_open** initializes reading from a datalist tree. The string **path* is the path to the top level datalist file to be opened. The value *look_processed* indicates whether the datalist parsing should look for or ignore processed data files (see the **mbprocess** and **mbdatalist** manual pages). The input values are:

**path*: datalist file to be opened
look_processed: processed file behavior
 0 : unset
 1 : ignore processed files
 2 : return processed files

The return values are:

***datalist_ptr*: pointer to datalist
 structure
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_datalist_read(
    int verbose,
    void *datalist_ptr,
    char *path,
    char *dpath,
    int *format,
    double *weight,
    int *error);
```

The function **mb_datalist_read** reads from a datalist tree, attempting to return the path to the next valid swath data file, the corresponding data format id, and a gridding weight (see the **mbprocess** and **mbdatalist** manual pages). If a processed version of the data file exists and processed files are to be used, **path* is returned as the path to the processed file. Information about the datalist tree is embedded in a data structure pointed to by **datalist_ptr*. The input values are:

**datalist_ptr*: pointer to datalist
 structure

The return values are:

**path*: swath data file (or its processed
 equivalent, if used)
**dpath*: path of the datalist file referencing
 **path*

**format*: MBIO format id
weight*: **mbgrid gridding weight
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures. This function is implemented as a call to **mb_datalist_read3**.

```

-----
int mb_datalist_readorg(
    int verbose,
    void *datalist_ptr,
    char *path,
    int *format,
    double *weight,
    int *error);
  
```

The function **mb_datalist_readorg** reads from a datalist tree exactly as **mb_datalist_read** does, except that the original (unprocessed) file path is always returned in **path*, even when a processed version of the file exists.

```

-----
int mb_datalist_read2(
    int verbose,
    void *datalist_ptr,
    int *pstatus,
    char *path,
    char *ppath,
    char *dpath,
    int *format,
    double *weight,
    int *error);
  
```

The function **mb_datalist_read2** reads from a datalist tree, returning both the original file path **path* and, separately, the path **ppath* of the processed version of that file (if one exists). The value **pstatus* indicates whether a processed file exists (**MB_PROCESSED_EXIST**) and if so whether it should be used (**MB_PROCESSED_USE**) or not (**MB_PROCESSED_NONE**). **dpath* is returned as the path of the datalist file that references **path*. **mb_datalist_read** is implemented as a call to **mb_datalist_read3**, which is in turn implemented as a call to **mb_datalist_read2** with the addition of alternative navigation file handling.

```

-----
int mb_datalist_read3(
    int verbose,
    void *datalist_ptr,
    int *pstatus,
    char *path,
    char *ppath,
    int *astatus,
    char *apath,
    char *dpath,
    int *format,
    double *weight,
    int *error);
  
```

The function **mb_datalist_read3** reads from a datalist tree exactly as **mb_datalist_read2** does, and in

addition returns the path **apath* to an alternative navigation file associated with the data file, if one is specified in the datalist. The value **astatus* indicates whether an alternative navigation file was found (**MB_ALTNV_USE**) or not (**MB_ALTNV_NONE**).

```
-----
int mb_datalist_recursion(
    int verbose,
    void *datalist_ptr,
    bool print,
    int *recursion,
    int *error);
```

The function **mb_datalist_recursion** returns the current recursion depth of the datalist tree rooted at **datalist_ptr* (datalist files may reference other datalist files, and this value tracks how deeply nested the currently open file is). If *print* is true, a summary of the recursive datalist structure is printed to standard error.

```
-----
int mb_datalist_close(
    int verbose,
    void **datalist_ptr,
    int *error);
```

The function **mb_datalist_close** closes an open datalist tree, and deallocates the data structure pointed to by **datalist_ptr*. The input values are:

**datalist_ptr*: pointer to datalist
structure

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
-----
int mb_imagelist_open(
    int verbose,
    void **imagelist_ptr,
    char *path,
    int *error);
```

The function **mb_imagelist_open** initializes reading from an imagelist tree in the same manner that **mb_datalist_open** does for a datalist tree. An imagelist references still or video camera image files (and, for stereo cameras, pairs of image files) associated with a survey, along with per-image navigation and timing metadata used when merging images with swath data (see the **mbphotomosaic** manual page). The string **path* is the path to the top level imagelist file to be opened. The return values are:

***imagelist_ptr*: pointer to imagelist
structure

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
-----
int mb_imagelist_read(
    int verbose,
    void *imagelist_ptr,
```



```

    int *imagestatus,
    bool *rectified,
    char *path0,
    char *path1,
    char *dpath,
    double *time_d0,
    double *time_d1,
    double *gain0,
    double *gain1,
    double *exposure0,
    double *exposure1,
    int *error);

```

The function **mb_imagelist_read** reads from an imagelist tree, attempting to return the path (or, for a stereo pair, paths) to the next image file along with its timestamp(s), camera gain(s), and exposure time(s). The value **imagestatus* indicates the kind of image record obtained (e.g. single image or stereo pair), and **rectified* indicates whether the image(s) have already been rectified. The return values are:

```

    *path0:      path of the (first) image file
    *path1:      path of the second image file of a
                  stereo pair, if any
    *dpath:      path of the imagelist file referencing
                  *path0 and *path1
    *time_d0:     time stamp of first image
    *time_d1:     time stamp of second image, if any
    *gain0:       camera gain for first image
    *gain1:       camera gain for second image, if any
    *exposure0:   exposure time for first image
    *exposure1:   exposure time for second image, if any
    error:        error value

```

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```

-----
int mb_imagelist_recursion(
    int verbose,
    void *imagelist_ptr,
    bool print,
    int *recursion,
    int *error);

```

The function **mb_imagelist_recursion** returns the current recursion depth of the imagelist tree rooted at **imagelist_ptr*, in the same manner that **mb_datalist_recursion** does for a datalist tree. If *print* is true, a summary of the recursive imagelist structure is printed to standard error.

```

-----
int mb_imagelist_close(
    int verbose,
    void **imagelist_ptr,
    int *error);

```

The function **mb_imagelist_close** closes an open imagelist tree, and deallocates the data structure pointed to by **imagelist_ptr*. A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
-----
int mb_check_info(
    int verbose,
    char *file,
    int lonflip,
    double bounds[4],
    bool *file_in_bounds,
    int *error);
```

The function **mb_check_info** checks whether the swath data file *file* has any data within the region specified by *lonflip* and *bounds*, without opening or reading the data file itself. This is done by reading the **mbinfo** summary statistics from the associated *.inf* file (see the **mbinfo** and **mbdatalist** manual pages), which lists the geographic bounds of the data. If *file* is "stdin" or no *.inf* file exists, **file_in_bounds* is simply set true. This function is used by **mbdatalist** and other programs to quickly exclude files with no data in a region of interest without opening every file in a large datalist.

```
-----
bool mb_should_make_fbt(
    int verbose,
    int format);
```

The function **mb_should_make_fbt** returns true if swath data files of the given **MBIO** *format* benefit from having an associated fast bathymetry file (*.fbt* suffix; see **mb_get_fbt** below) generated for them, and false for formats (e.g. navigation-only or ASCII formats) for which a *.fbt* file is not useful.

```
-----
bool mb_should_make_fnv(
    int verbose,
    int format);
```

The function **mb_should_make_fnv** returns true if swath data files of the given **MBIO** *format* benefit from having an associated fast navigation file (*.fnv* suffix; see **mb_get_fnv** below) generated for them, and false otherwise.

```
-----
int mb_make_info(
    int verbose,
    bool force,
    char *file,
    int format,
    int *error);
```

The function **mb_make_info** checks the modification times of the *.inf*, *.fbt*, and *.fnv* ancillary files associated with the swath data file *file* against the modification time of *file* itself, and regenerates any that are missing or out of date by invoking **mbinfo** (for the *.inf* file) and **mbcopy** (for the *.fbt* and *.fnv* files) as subprocesses. If *force* is true, the ancillary files are regenerated unconditionally.

```
-----
int mb_make_info_datalist(
    int verbose,
    bool force,
    char *read_file,
    int *format,
```

```
int *error);
```

The function **mb_make_info_datalist** calls **mb_make_info** for every swath data file referenced (directly or recursively) by the datalist or single swath file named *read_file*.

```
-----
int mb_get_fbt(
    int verbose,
    char *file,
    int *format,
    int *error);
```

The function **mb_get_fbt** checks whether a fast bathymetry file (*file* with a *.fbt* suffix appended) exists and is present; if so **file* is replaced with the path of the *.fbt* file and **format* is set to the fast bathymetry format id (71). If no *.fbt* file exists, **file* and **format* are left unchanged. This allows an application such as **mbgrid** to transparently read the much smaller and faster *.fbt* file in place of the original data file when one is available.

```
-----
int mb_get_fnv(
    int verbose,
    char *file,
    int *format,
    int *error);
```

The function **mb_get_fnv** behaves exactly as **mb_get_fbt** does, except that it looks for a fast navigation file (a *.fnv* suffix) and, if found, sets **format* to the fast navigation format id (166).

```
-----
int mb_get_ffa(
    int verbose,
    char *file,
    int *format,
    int *error);
```

The function **mb_get_ffa** behaves exactly as **mb_get_fbt** does, except that it looks for a fast amplitude file (a *.ffa* suffix) and, if found, sets **format* to the fast bathymetry/amplitude format id (71).

```
-----
int mb_get_ffs(
    int verbose,
    char *file,
    int *format,
    int *error);
```

The function **mb_get_ffs** behaves exactly as **mb_get_fbt** does, except that it looks for a fast sidescan file (a *.ffs* suffix) and, if found, sets **format* to the fast bathymetry/sidescan format id (71).

```
-----
int mb_swathbounds(
    int verbose,
    int checkgood,
    int nbath,
    int nss,
```

```

char *beamflag,
double *bathacrosstrack,
double *ss,
double *ssacrosstrack,
int *ibeamport,
int *ibeamcntr,
int *ibeamstbd,
int *ipixelport,
int *ipixelcntr,
int *ipixelstbd,
int *error);

```

The function **mb_swathbounds** examines the bathymetry and sidescan acrosstrack distance arrays of a ping (of *nbath* beams and *nss* pixels) and returns the port-most, center, and starboard-most beam and pixel indices. If *checkgood* is set, beams and pixels flagged bad are excluded from consideration. This is used by plotting and processing programs (e.g. **mbedit** and **mbeditviz**) to quickly determine the acrosstrack extent of a ping's good data.

```

-----
int mb_info_init(
    int verbose,
    struct mb_info_struct *mb_info,
    int *error);

```

The function **mb_info_init** initializes (zeroes) the fields of the *struct mb_info_struct* pointed to by *mb_info*, the data structure used by **mb_get_info** to hold the summary statistics parsed from an *.inf* file.

```

-----
int mb_get_info(
    int verbose,
    char *file,
    struct mb_info_struct *mb_info,
    int lonflip,
    int *error);

```

The function **mb_get_info** reads the **mbinfo** summary statistics for the swath data file *file* from its associated *.inf* file (generating that file first with **mb_make_info** if necessary) and returns them in the *struct mb_info_struct* pointed to by *mb_info*. The *lonflip* value controls how longitude values are wrapped when read from the *.inf* file.

```

-----
int mb_get_info_datalist(
    int verbose,
    char *read_file,
    int *format,
    struct mb_info_struct *mb_info,
    int lonflip,
    int *error);

```

The function **mb_get_info_datalist** behaves as **mb_get_info** does, but accepts a datalist as well as a single swath file for *read_file*; when given a datalist, the summary statistics returned in *mb_info* are accumulated over every swath file referenced by the datalist.

```
-----
int mb_alloc(
    int verbose,
    void *mbio_ptr,
    void **store_ptr,
    int *error);
```

The function **mb_alloc** allocates a data structure for internal storage of swath sonar data and returns a pointer to this structure in *store_ptr*. The data structure is specific to the data format identified in the **MBIO** data structure pointed to by *mbio_ptr*. The input values are:

mbio_ptr: pointer to **MBIO** structure

The return values are:

store_ptr: pointer to storage data
structure

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
-----
int mb_deall(
    int verbose,
    void *mbio_ptr,
    void **store_ptr,
    int *error);
```

The function **mb_deall** deallocates a format specific swath sonar data structure pointed to by *store_ptr*. The input values are:

mbio_ptr: pointer to **MBIO** structure

store_ptr: pointer to storage data
structure

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
-----
int mb_malloc(
    int verbose,
    size_t size,
    void **ptr,
    int *error);
```

The function **mb_malloc** allocates *size* bytes of memory, returning the new pointer in *ptr*, in the same manner as the standard C *malloc()* function, but also optionally tracking the allocation (see **mb_mem_list_enable**) and printing debug messages when *verbose* is high enough or debugging has been turned on with **mb_mem_debug_on**.

```
-----
int mb_realloc(
    int verbose,
    size_t size,
    void **ptr,
    int *error);
```

The function **mb_realloc** resizes the memory block pointed to by **ptr* (allocated by **mb_malloc** or **mb_realloc**) to *size* bytes, in the same manner as the standard C *realloc()* function, updating **ptr* and any memory tracking as **mb_malloc** does.

```
-----
int mb_free(
    int verbose,
    void **ptr,
    int *error);
```

The function **mb_free** frees the memory pointed to by **ptr* (allocated by **mb_malloc** or **mb_realloc**), sets **ptr* to NULL, and updates any memory tracking as **mb_malloc** does.

```
-----
int mb_mallocd(
    int verbose,
    const char *sourcefile,
    int sourceline,
    size_t size,
    void **ptr,
    int *error);
```

The function **mb_mallocd** behaves exactly as **mb_malloc** does, but also records the *sourcefile* name and *sourceline* number of the call site in the memory tracking list (when enabled), to aid in locating memory leaks. **MB-System** source code normally invokes this indirectly via the **mb_mallocd** macro, which supplies **__FILE__** and **__LINE__** automatically.

```
-----
int mb_reallocd(
    int verbose,
    const char *sourcefile,
    int sourceline,
    size_t size,
    void **ptr,
    int *error);
```

The function **mb_reallocd** behaves exactly as **mb_realloc** does, but also records the *sourcefile* name and *sourceline* number of the call site, in the same manner as **mb_mallocd**.

```
-----
int mb_freed(
    int verbose,
    const char *sourcefile,
    int sourceline,
    void **ptr,
    int *error);
```

The function **mb_freed** behaves exactly as **mb_free** does, but also records the *sourcefile* name and *sourceline* number of the call site, in the same manner as **mb_mallocd**.

```
-----
int mb_mem_list_enable(
    int verbose,
```

```
int *error);
```

The function **mb_mem_list_enable** turns on internal tracking, by **mb_malloc/mb_realloc/mb_free** and related functions, of a list of all currently allocated memory blocks, enabling the use of **mb_memory_status** and **mb_memory_list**.

```
-----
int mb_mem_list_disable(
    int verbose,
    int *error);
```

The function **mb_mem_list_disable** turns off the memory allocation tracking enabled by **mb_mem_list_enable**.

```
-----
int mb_mem_debug_on(
    int verbose,
    int *error);
```

The function **mb_mem_debug_on** turns on printing of debug messages by the memory allocation functions above regardless of the *verbose* level passed to them.

```
-----
int mb_mem_debug_off(
    int verbose,
    int *error);
```

The function **mb_mem_debug_off** turns off the unconditional debug message printing enabled by **mb_mem_debug_on**.

```
-----
int mb_memory_clear(
    int verbose,
    int *error);
```

The function **mb_memory_clear** frees every memory block currently in the allocation list maintained when memory tracking is enabled (see **mb_mem_list_enable**), and empties the list.

```
-----
int mb_memory_status(
    int verbose,
    int *nalloc,
    int *nallocmax,
    int *overflow,
    size_t *allocsize,
    int *error);
```

The function **mb_memory_status** returns, when memory tracking is enabled (see **mb_mem_list_enable**), the number of memory blocks currently allocated (**nalloc*), the maximum number of blocks the tracking list can hold (**nallocmax*), a flag (**overflow*) indicating whether that maximum has been exceeded (in which case tracking becomes incomplete), and the total number of bytes currently allocated (**allocsize*).

```
int mb_memory_list(
    int verbose,
    int *error);
```

The function **mb_memory_list** prints to standard error a listing of every memory block currently in the allocation tracking list (see **mb_mem_list_enable**), including the source file and line number recorded by **mb_mallocd**.

```
-----
int mb_update_arrays(
    int verbose,
    void *mbio_ptr,
    int nbath,
    int namp,
    int nss,
    int *error);
```

The function **mb_update_arrays** resizes every array registered with **mb_register_array** for the **MBIO** descriptor pointed to by *mbio_ptr* to match new maximum numbers of bathymetry beams *nbath*, amplitude beams *namp*, and sidescan pixels *nss*. It is called internally as necessary while reading data whose numbers of beams or pixels can vary from ping to ping.

```
-----
int mb_update_arrayptr(
    int verbose,
    void *mbio_ptr,
    void **handle,
    int *error);
```

The function **mb_update_arrayptr** updates the stored copy of one registered array pointer **handle* for the **MBIO** descriptor pointed to by *mbio_ptr*, for use after the calling program has reallocated that array itself.

```
-----
int mb_list_arrays(
    int verbose,
    void *mbio_ptr,
    int *error);
```

The function **mb_list_arrays** prints to standard error a listing of every array registered with **mb_register_array** for the **MBIO** descriptor pointed to by *mbio_ptr*.

```
-----
int mb_deall_ioarrays(
    int verbose,
    void *mbio_ptr,
    int *error);
```

The function **mb_deall_ioarrays** frees every array registered with **mb_register_array** for the **MBIO** descriptor pointed to by *mbio_ptr*. It is called internally by **mb_close**.

```
-----
int mb_get_time(
    int verbose,
```



```

    int time_i[7],
    double *time_d);

```

The function **mb_get_time** converts the seven-element time array *time_i* (year, month, day, hour, minute, second, microsecond) into **time_d*, the number of seconds since January 1, 1970 (with no time zone correction applied - **MB-System** always works in whatever time standard was used when the data were logged). Leap days are accounted for; leap seconds are not.

```

-----
int mb_get_date(
    int verbose,
    double time_d,
    int time_i[7]);

```

The function **mb_get_date** is the inverse of **mb_get_time**, converting *time_d* seconds since January 1, 1970 into the seven-element time array *time_i*.

```

-----
int mb_get_date_string(
    int verbose,
    double time_d,
    char *string);

```

The function **mb_get_date_string** converts *time_d* seconds since January 1, 1970 into a human-readable date and time *string*.

```

-----
int mb_get_jtime(
    int verbose,
    int time_i[7],
    int time_j[5]);

```

The function **mb_get_jtime** converts the seven-element time array *time_i* into the five-element Julian day time array *time_j* (year, Julian day of year, hour, minute, second - with *time_j*[4] tracking the residual microseconds), the alternate time representation used internally by some sonar systems and data formats.

```

-----
int mb_get_itime(
    int verbose,
    int time_j[5],
    int time_i[7]);

```

The function **mb_get_itime** is the inverse of **mb_get_jtime**, converting the five-element Julian day time array *time_j* into the seven-element time array *time_i*.

```

-----
char *mb_day_name(
    int verbose,
    int day);

```

The function **mb_day_name** returns a pointer to a static string naming the day of the week numbered 1 (Sunday) through 7 (Saturday) by *day*; any other value returns "Unknown".

```
-----
char *mb_month_name(
    int verbose,
    int month);
```

The function **mb_month_name** returns a pointer to a static string naming the month numbered 1 (January) through 12 (December) by *month*; any other value returns "Unknown".

```
-----
int mb_fix_y2k(
    int verbose,
    int year_short,
    int *year_long);
```

The function **mb_fix_y2k** converts a two digit year value *year_short* into a four digit year **year_long*, adding 1900 if *year_short* is 62 or greater and 2000 otherwise (no digital swath sonar data predates 1962, when multi-beam sonars were first patented and built).

```
-----
int mb_unfix_y2k(
    int verbose,
    int year_long,
    int *year_short);
```

The function **mb_unfix_y2k** is the inverse of **mb_fix_y2k**, converting a four digit year *year_long* into a two digit year **year_short*.

```
-----
int mb_error(
    int verbose,
    int error,
    char **message);
```

Given the error value *error*, **mb_error** returns a short error message in the string ***message*. The *verbose* value controls the standard error output verbosity of the function. The return status value signals success if *error* is a recognized **MBIO** error value and failure otherwise.

```
-----
int mb_notice_log_datatype(
    int verbose,
    void *mbio_ptr,
    int data_id);
```

The function **mb_notice_log_datatype** increments the count, in the notice list maintained in the **MBIO** descriptor pointed to by *mbio_ptr*, of data records of kind *data_id* that have been read or written. This supports the end-of-run notice summary printed by many **MB-System** programs.

```
-----
int mb_notice_log_error(
    int verbose,
    void *mbio_ptr,
    int error_id);
```

The function **mb_notice_log_error** behaves as **mb_notice_log_datatype** does, but increments the count of occurrences of the (nonfatal) error condition *error_id*.

```
-----
int mb_notice_log_problem(
    int verbose,
    void *mbio_ptr,
    int problem_id);
```

The function **mb_notice_log_problem** behaves as **mb_notice_log_datatype** does, but increments the count of occurrences of the data problem condition *problem_id* (e.g. a recognized but questionable value encountered while reading).

```
-----
int mb_notice_get_list(
    int verbose,
    void *mbio_ptr,
    int *notice_list);
```

The function **mb_notice_get_list** copies the notice list maintained in the **MBIO** descriptor pointed to by *mbio_ptr* (as tallied by **mb_notice_log_datatype**, **mb_notice_log_error**, and **mb_notice_log_problem**) into the array *notice_list*, which must be allocated by the calling program to hold **MB_NOTICE_MAX** values.

```
-----
int mb_notice_message(
    int verbose,
    int notice,
    char **message);
```

The function **mb_notice_message** returns, in the string ***message*, a short description of the notice list entry *notice* (one of the data record, error, or problem identifiers logged by the **mb_notice_log_*** functions above).

```
-----
int mb_navint_add(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double lon_easting,
    double lat_northing,
    int *error);
```

The function **mb_navint_add** adds a navigation fix to a circular buffer of navigation values maintained in the **MBIO** data structure pointed to by **mbio_ptr*. This buffer is used to interpolate navigation for data formats where the navigation is asynchronous (where navigation and survey pings come in different data records). The input values are:

<i>*mbio_ptr</i> :	pointer to MBIO structure
<i>time_d</i> :	time of navigation fix in seconds since 1/1/70 00:00:00
<i>lon_easting</i> :	longitude (degrees), or projected easting if the MBIO structure has been set to use a projected coordinate system
<i>lat_northing</i> :	latitude (degrees), or projected

nothing if the **MBIO** structure
has been set to use a projected
coordinate system

The return values are:

error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_navint_interp(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double heading,
    double rawspeed,
    double *lon,
    double *lat,
    double *speed,
    int *error);
```

The function **mb_navint_interp** interpolates navigation to the time *time_d* using a circular buffer of navigation values maintained in the **MBIO** data structure pointed to by **mbio_ptr*. This buffer is used to interpolate navigation for data formats where the navigation is asynchronous (where navigation and survey pings come in different data records). The input values are:

mbio_ptr*: pointer to **MBIO structure
time_d: time of current ping in seconds
 since 1/1/70 00:00:00
heading: heading in degrees
rawspeed: speed in km/hr
 (zero if not known)

The return values are:

**lon*: longitude (degrees)
**lat*: latitude (degrees)
**speed*: speed made good in km/hr
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_navint_prjinterp(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double heading,
    double rawspeed,
    double *easting,
    double *northing,
    double *speed,
    int *error);
```

The function **mb_navint_prjinterp** behaves exactly as **mb_navint_interp** does, except that it interpolates and returns projected easting and northing values (**easting*, **northing*) rather than longitude and latitude, for use when the **MBIO** structure has been set to work in a projected coordinate system.

```
int mb_attint_add(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double heave,
    double roll,
    double pitch,
    int *error);
```

The function **mb_attint_add** adds an attitude (heave, roll, pitch) data point to a circular buffer of attitude values maintained in the **MBIO** data structure pointed to by ***mbio_ptr**. This buffer is used to interpolate attitude for data formats where the attitude is asynchronous (where attitude and survey pings come in different data records). The input values are:

<i>*mbio_ptr</i> :	pointer to MBIO structure
<i>time_d</i> :	time of attitude in seconds since 1/1/70 00:00:00
<i>heave</i> :	heave (meters, up +)
<i>roll</i> :	roll (degrees, starboard up +)
<i>pitch</i> :	pitch (degrees, forward up +)

The return values are:

<i>error</i> :	error value
----------------	-------------

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
int mb_attint_nadd(
    int verbose,
    void *mbio_ptr,
    int nsamples,
    double *time_d,
    double *heave,
    double *roll,
    double *pitch,
    int *error);
```

The function **mb_attint_nadd** adds *nsamples* attitude (heave, roll, pitch) data points at once to the circular buffer described under **mb_attint_add** above, which is more efficient than calling **mb_attint_add** repeatedly when a block of attitude samples is available at once (e.g. when merging externally logged attitude data).

```
int mb_attint_interp(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double *heave,
    double *roll,
    double *pitch,
    int *error);
```

The function **mb_attint_interp** interpolates attitude (heave, roll, pitch) data to the time *time_d* using a circular buffer of attitude values maintained in the **MBIO** data structure pointed to by ***mbio_ptr**. This buffer is used to interpolate attitude for data formats where the attitude is asynchronous (where attitude and survey

pings come in different data records). The input values are:

mbio_ptr*: pointer to **MBIO structure
time_d: time of current ping in seconds
 since 1/1/70 00:00:00

The return values are:

**heave*: heave (meters, up +)
**roll*: roll (degrees, starboard up +)
**pitch*: pitch (degrees, forward up +)
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
-----
int mb_depint_add(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double sensordepth,
    int *error);
```

The function **mb_depint_add** adds a sensor (transducer) depth value to a circular buffer of sensor depth values maintained in the **MBIO** data structure pointed to by **mbio_ptr*, in the same manner that **mb_hedint_add** does for heading.

```
-----
int mb_depint_interp(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double *sensordepth,
    int *error);
```

The function **mb_depint_interp** interpolates sensor depth to the time *time_d* using the circular buffer maintained by **mb_depint_add**, in the same manner that **mb_hedint_interp** does for heading.

```
-----
int mb_altint_add(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double altitude,
    int *error);
```

The function **mb_altint_add** adds an altitude value to a circular buffer of altitude values maintained in the **MBIO** data structure pointed to by **mbio_ptr*, in the same manner that **mb_hedint_add** does for heading.

```
-----
int mb_altint_interp(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double *altitude,
    int *error);
```

The function **mb_altint_interp** interpolates altitude to the time *time_d* using the circular buffer maintained by **mb_altint_add**, in the same manner that **mb_hedint_interp** does for heading.

```
-----
int mb_hedint_add(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double heading,
    int *error);
```

The function **mb_hedint_add** adds a heading point to a circular buffer of heading values maintained in the **MBIO** data structure pointed to by ***mbio_ptr**. This buffer is used to interpolate heading for data formats where the heading is asynchronous (where heading and survey pings come in different data records). The input values are:

<i>*mbio_ptr</i> :	pointer to MBIO structure
<i>time_d</i> :	time of heading value in seconds since 1/1/70 00:00:00
<i>heading</i> :	heading (degrees)

The return values are:

<i>error</i> :	error value
----------------	-------------

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
-----
int mb_hedint_nadd(
    int verbose,
    void *mbio_ptr,
    int nsamples,
    double *time_d,
    double *heading,
    int *error);
```

The function **mb_hedint_nadd** adds *nsamples* heading values at once to the circular buffer described under **mb_hedint_add** above, which is more efficient than calling **mb_hedint_add** repeatedly when a block of heading samples is available at once.

```
-----
int mb_hedint_interp(
    int verbose,
    void *mbio_ptr,
    double time_d,
    double *heading,
    int *error);
```

The function **mb_hedint_interp** interpolates heading to the time *time_d* using a circular buffer of heading values maintained in the **MBIO** data structure pointed to by ***mbio_ptr**. This buffer is used to interpolate heading for data formats where the heading is asynchronous (where heading and survey pings come in different data records). The input values are:

<i>*mbio_ptr</i> :	pointer to MBIO structure
<i>time_d</i> :	time of current ping in seconds since 1/1/70 00:00:00

The return values are:

**heading:* heading in degrees
error: error value

A status value indicating success or failure is returned; the error value argument *error* passes more detailed information about failures.

```
-----
int mb_loadnavdata(
    int verbose,
    char *merge_nav_file,
    int merge_nav_format,
    int merge_nav_lonflip,
    int *merge_nav_num,
    int *merge_nav_alloc,
    double **merge_nav_time_d,
    double **merge_nav_lon,
    double **merge_nav_lat,
    double **merge_nav_speed,
    int *error);
```

The function **mb_loadnavdata** reads an entire external navigation file *merge_nav_file* (of **MBIO** format *merge_nav_format*, applying *merge_nav_lonflip*) into time, longitude, latitude, and speed arrays that are allocated (or reallocated, tracked by **merge_nav_alloc*) and returned as **merge_nav_time_d*, **merge_nav_lon*, **merge_nav_lat*, and **merge_nav_speed*, with **merge_nav_num* set to the number of values read. This is used by programs such as **mbprocess** to merge externally logged navigation into swath data using **mb_linear_interp** or **mb_linear_interp_longitude** and **mb_linear_interp_latitude**.

```
-----
int mb_loadsensordepthdata(
    int verbose,
    char *merge_sensordepth_file,
    int merge_sensordepth_format,
    int *merge_sensordepth_num,
    int *merge_sensordepth_alloc,
    double **merge_sensordepth_time_d,
    double **merge_sensordepth_sensordepth,
    int *error);
```

The function **mb_loadsensordepthdata** behaves as **mb_loadnavdata** does, but reads an external sensor depth time series into time and sensor depth arrays.

```
-----
int mb_loadaltitudedata(
    int verbose,
    char *merge_altitude_file,
    int merge_altitude_format,
    int *merge_altitude_num,
    int *merge_altitude_alloc,
    double **merge_altitude_time_d,
    double **merge_altitude_altitude,
    int *error);
```

The function **mb_loadaltitudedata** behaves as **mb_loadnavdata** does, but reads an external altitude time series into time and altitude arrays.


```

-----
int mb_loadheadingdata(
    int verbose,
    char *merge_heading_file,
    int merge_heading_format,
    int *merge_heading_num,
    int *merge_heading_alloc,
    double **merge_heading_time_d,
    double **merge_heading_heading,
    int *error);

```

The function **mb_loadheadingdata** behaves as **mb_loadnavdata** does, but reads an external heading time series into time and heading arrays.

```

-----
int mb_loadattitudedata(
    int verbose,
    char *merge_attitude_file,
    int merge_attitude_format,
    int *merge_attitude_num,
    int *merge_attitude_alloc,
    double **merge_attitude_time_d,
    double **merge_attitude_roll,
    double **merge_attitude_pitch,
    double **merge_attitude_heave,
    int *error);

```

The function **mb_loadattitudedata** behaves as **mb_loadnavdata** does, but reads an external attitude time series into time, roll, pitch, and heave arrays.

```

-----
int mb_loadsoundspeeddata(
    int verbose,
    char *merge_soundspeed_file,
    int merge_soundspeed_format,
    int *merge_soundspeed_num,
    int *merge_soundspeed_alloc,
    double **merge_soundspeed_time_d,
    double **merge_soundspeed_soundspeed,
    int *error);

```

The function **mb_loadsoundspeeddata** behaves as **mb_loadnavdata** does, but reads an external near-surface sound speed time series into time and sound speed arrays.

```

-----
int mb_loadtimeshiftdata(
    int verbose,
    char *merge_timeshift_file,
    int merge_timeshift_format,
    int *merge_timeshift_num,
    int *merge_timeshift_alloc,
    double **merge_timeshift_time_d,
    double **merge_timeshift_timeshift,

```

```
int *error);
```

The function **mb_loadtimeshiftdata** behaves as **mb_loadnavdata** does, but reads an external clock time-shift correction time series into time and timeshift arrays.

```
-----
int mb_apply_time_latency(
    int verbose,
    int data_num,
    double *data_time_d,
    int time_latency_mode,
    double time_latency_static,
    int time_latency_num,
    double *time_latency_time_d,
    double *time_latency_value,
    int *error);
```

The function **mb_apply_time_latency** corrects, in place, the *data_num* timestamps in *data_time_d* for sensor time latency (clock lag). If *time_latency_mode* is **MB_SENSOR_TIME_LATENCY_STATIC**, a constant *time_latency_static* offset (seconds) is subtracted from every timestamp. If *time_latency_mode* is **MB_SENSOR_TIME_LATENCY_MODEL**, a time-varying latency is instead interpolated from the *time_latency_num*-point model given by *time_latency_time_d* and *time_latency_value* and subtracted from each timestamp.

```
-----
int mb_apply_time_filter(
    int verbose,
    int data_num,
    double *data_time_d,
    double *data_value,
    double filter_length,
    int *error);
```

The function **mb_apply_time_filter** replaces, in place, the *data_num* values in *data_value* (timestamped by *data_time_d*) with a running-mean smoothed version computed over a time window of length *filter_length* seconds. This is used, for example, to smooth noisy externally logged sensor time series before merging them with swath data.

```
-----
int mb_get_double(
    double *value,
    char *str,
    int nchar);
```

The function **mb_get_double** parses the first *nchar* characters of the string **str* for a floating point value, storing this value as a double in **value*.

```
-----
int mb_get_int(
    int *value,
    char *str,
    int nchar);
```

The function **mb_get_int** parses the first *nchar* characters of the string **str* for an integer value, storing this value as a int in **value*.

```
int mb_get_binary_short(
    bool swapped,
    void *buffer,
    const void *ptr);
```

The function **mb_get_binary_short** extracts a short int value from the first two bytes pointed to by **buffer*, copying the (possibly byte-swapped) result into the location pointed to by **ptr* (cast internally to *short* *). If *swapped* is true, the byte order of the extracted value is swapped.

```
int mb_get_binary_int(
    bool swapped,
    void *buffer,
    const void *ptr);
```

The function **mb_get_binary_int** extracts an int value from the first four bytes pointed to by **buffer*, copying the (possibly byte-swapped) result into the location pointed to by **ptr* (cast internally to *int* *). If *swapped* is true, the byte order of the extracted value is swapped.

```
int mb_get_binary_float(
    bool swapped,
    void *buffer,
    const void *ptr);
```

The function **mb_get_binary_float** extracts a float value from the first four bytes pointed to by **buffer*, copying the (possibly byte-swapped) result into the location pointed to by **ptr* (cast internally to *float* *). If *swapped* is true, the byte order of the extracted value is swapped.

```
int mb_get_binary_double(
    bool swapped,
    void *buffer,
    const void *ptr);
```

The function **mb_get_binary_double** extracts a double value from the first eight bytes pointed to by **buffer*, copying the (possibly byte-swapped) result into the location pointed to by **ptr* (cast internally to *double* *). If *swapped* is true, the byte order of the extracted value is swapped.

```
int mb_get_binary_long(
    bool swapped,
    void *buffer,
    const void *ptr);
```

The function **mb_get_binary_long** extracts a 64 bit long integer value from the first eight bytes pointed to by **buffer*, copying the (possibly byte-swapped) result into the location pointed to by **ptr* (cast internally to *mb_s_long* *). If *swapped* is true, the byte order of the extracted value is swapped.

```
-----  
int mb_put_binary_short(  
    bool swapped,  
    short value,  
    void *buffer);
```

The function **mb_put_binary_short** inserts a short int value into the first two bytes pointed to by **buffer*. If *swapped* is true, the byte order of *value* is swapped.

```
-----  
int mb_put_binary_int(  
    bool swapped,  
    int value,  
    void *buffer);
```

The function **mb_put_binary_int** inserts an int value into the first four bytes pointed to by **buffer*. If *swapped* is true, the byte order of *value* is swapped.

```
-----  
int mb_put_binary_float(  
    bool swapped,  
    float value,  
    void *buffer);
```

The function **mb_put_binary_float** inserts a float value into the first four bytes pointed to by **buffer*. If *swapped* is true, the byte order of *value* is swapped.

```
-----  
int mb_put_binary_double(  
    bool swapped,  
    double value,  
    void *buffer);
```

The function **mb_put_binary_double** inserts a double value into the first eight bytes pointed to by **buffer*. If *swapped* is true, the byte order of *value* is swapped.

```
-----  
int mb_put_binary_long(  
    bool swapped,  
    mb_s_long value,  
    void *buffer);
```

The function **mb_put_binary_long** inserts a 64 bit long integer value into the first eight bytes pointed to by **buffer*. If *swapped* is true, the byte order of *value* is swapped.

```
-----  
int mb_swap_check(void);
```

The function **mb_swap_check** returns true if the current machine is little-endian (byte-swapped relative to the big-endian byte order used to store most **MBIO** binary data formats) and false if it is big-endian.

```
-----  
int mb_swap_float(
```

```
float *a);
```

The function **mb_swap_float** reverses the byte order of the float pointed to by *a*, in place.

```
-----
int mb_swap_double(
    double *a);
```

The function **mb_swap_double** reverses the byte order of the double pointed to by *a*, in place.

```
-----
int mb_swap_long(
    mb_s_long *a);
```

The function **mb_swap_long** reverses the byte order of the 64 bit long integer pointed to by *a*, in place.

```
-----
int mb_get_bounds(
    char *text,
    double *bounds);
```

The function **mb_get_bounds** parses the string **text* and extracts geographic bounds of a rectangular region in the form:

```
bounds[0]: minimum longitude
bounds[1]: maximum longitude
bounds[2]: minimum latitude
bounds[3]: maximum latitude
```

where **text* is in the standard **GMT** bounds form. The longitude and latitude values in **text* should be separated by a '/' character, and individual values may be represented in decimal degrees or in "dd:mm:ss" form (dd=degrees, mm=minutes, ss=seconds).

```
double mb_ddmmss_to_degree(
    char *text);
```

The function **mb_ddmmss_to_degree** parses the string **text* and extracts a decimal longitude or latitude value from a "dd:mm:ss" (dd=degrees, mm=minutes, ss=seconds) value.

```
-----
int mb_apply_lonflip(
    int verbose,
    int lonflip,
    double *longitude);
```

The function **mb_apply_lonflip** rewrites **longitude* in place so that it falls in the range required by *lonflip*: if *lonflip* is negative, **longitude* is wrapped into [-360,0]; if zero, into [-180,180]; if positive, into [0,360]. See **mb_read_init** for the general meaning of *lonflip*.

```
-----
int mb_coor_scale(
    int verbose,
    double latitude,
    double *mtodeglon,
    double *mtodeglat);
```

The function **mb_coor_scale** returns the scaling factors **mtodeglon* and **mtodeglat* that convert distances in meters to longitude and latitude differences in degrees at the given *latitude*, using a standard ellipsoidal earth model. These scale factors are used throughout **MB-System** to convert between geographic coordinates and local distances (for example, in computing sensor lever arm offsets).

```
-----
int mb_alvinxy_scale(
    int verbose,
    double latitude,
    double *mtodeglon,
    double *mtodeglat);
```

The function **mb_alvinxy_scale** returns scaling factors **mtodeglon* and **mtodeglat* in the same sense as **mb_coor_scale**, but computed using the Clark 1866 spheroid coefficients used by the WHOI Alvin group's "AlvinXY" local rectilinear navigation coordinate frame convention (Murphy and Singh, 2010).

```
-----
int mb_proj_init(
    int verbose,
    char *projection,
    void **pjptr,
    int *error);
```

The function **mb_proj_init** initializes, using the **libproj** cartographic projections library, a map projection described by *projection* (a **PROJ/EPSSG** projection specification string, as also accepted by the **-J** options of many **MB-System** and **GMT** programs) and returns a pointer to the initialized projection in **pjptr* for use by **mb_proj_forward** and **mb_proj_inverse**.

```
-----
int mb_proj_free(
    int verbose,
    void **pjptr,
    int *error);
```

The function **mb_proj_free** deallocates a projection previously initialized by **mb_proj_init** and pointed to by **pjptr*.

```
-----
int mb_proj_forward(
    int verbose,
    void *pjptr,
    double lon,
    double lat,
    double *easting,
    double *northing,
    int *error);
```

The function **mb_proj_forward** applies the forward map projection *pjptr* (initialized by **mb_proj_init**) to convert geographic longitude *lon* and latitude *lat* (degrees) into projected **easting* and **northing* (meters).

```
-----
int mb_proj_inverse(
    int verbose,
```

```

void *pjptr,
double easting,
double northing,
double *lon,
double *lat,
int *error);

```

The function **mb_proj_inverse** applies the inverse of the map projection *pjptr* (initialized by **mb_proj_init**) to convert projected *easting* and *northing* (meters) back into geographic **lon* and **lat* (degrees).

```

-----
int mb_spline_init(
    int verbose,
    const double *x,
    const double *y,
    int n,
    double yp1,
    double ypn,
    double *y2,
    int *error);

```

The function **mb_spline_init** computes the second-derivative array **y2* of a natural cubic spline through the *n* data points (*x[i]*, *y[i]*), given the first-derivative boundary conditions *yp1* and *ypn* at the two endpoints (following Press et al., *Numerical Recipes in C*). The array **y2* is then passed to **mb_spline_interp** to evaluate the spline at arbitrary points.

```

-----
int mb_spline_interp(
    int verbose,
    const double *xa,
    const double *ya,
    double *y2a,
    int n,
    double x,
    double *y,
    int *i,
    int *error);

```

The function **mb_spline_interp** evaluates, at the point *x*, the cubic spline defined by the *n* data points (*xa[i]*, *ya[i]*) and the second-derivative array *y2a* computed by **mb_spline_init**, returning the interpolated value in **y*. The index **i* should be initialized to zero before the first call and is thereafter maintained across successive calls to speed up the table lookup when *x* values are supplied in increasing order.

```

-----
int mb_linear_interp(
    int verbose,
    const double *xa,
    const double *ya,
    int n,
    double x,
    double *y,
    int *i,
    int *error);

```

The function **mb_linear_interp** evaluates, at the point x , simple linear interpolation (or, outside the data range, extrapolation using the nearest endpoint) through the n data points ($xa[i]$, $ya[i]$), returning the interpolated value in $*y$. The index $*i$ is used in the same way as in **mb_spline_interp**.

```
-----
int mb_linear_interp_longitude(
    int verbose,
    const double *xa,
    const double *ya,
    int n,
    double x,
    double *y,
    int *i,
    int *error);
```

The function **mb_linear_interp_longitude** behaves as **mb_linear_interp** does, but treats the ya values as longitudes, so that interpolation is done correctly across the ± 180 degree dateline discontinuity.

```
-----
int mb_linear_interp_latitude(
    int verbose,
    const double *xa,
    const double *ya,
    int n,
    double x,
    double *y,
    int *i,
    int *error);
```

The function **mb_linear_interp_latitude** behaves as **mb_linear_interp** does, but treats the ya values as latitudes, without extrapolating beyond ± 90 degrees.

```
-----
int mb_linear_interp_heading(
    int verbose,
    const double *xa,
    const double *ya,
    int n,
    double x,
    double *y,
    int *i,
    int *error);
```

The function **mb_linear_interp_heading** behaves as **mb_linear_interp** does, but treats the ya values as headings, so that interpolation is done correctly across the $0/360$ degree discontinuity.

```
-----
int mb_takeoff_to_rollpitch(
    int verbose,
    double theta,
    double phi,
    double *alpha,
    double *beta,
```



```
int *error);
```

The function **mb_takeoff_to_rollpitch** translates angles from the "takeoff" coordinate reference frame to the "rollpitch" coordinate system. See the discussion of coordinate systems below.

```
-----
int mb_rollpitch_to_takeoff(
    int verbose,
    double alpha,
    double beta,
    double *theta,
    double *phi,
    int *error);
```

The function **mb_rollpitch_to_takeoff** translates angles from the "rollpitch" coordinate reference frame to the "takeoff" coordinate system. See the discussion of coordinate systems below.

```
-----
int mb_xyz_to_takeoff(
    int verbose,
    double x,
    double y,
    double z,
    double *theta,
    double *phi,
    int *error);
```

The function **mb_xyz_to_takeoff** converts a Cartesian beam direction vector (x, y, z) into the "takeoff" coordinate system angles **theta* (from vertical) and **phi* (from the alongtrack direction). See the discussion of coordinate systems below.

```
-----
int mb_lever(
    int verbose,
    double sonar_offset_x,
    double sonar_offset_y,
    double sonar_offset_z,
    double nav_offset_x,
    double nav_offset_y,
    double nav_offset_z,
    double vru_offset_x,
    double vru_offset_y,
    double vru_offset_z,
    double vru_pitch,
    double vru_roll,
    double *lever_x,
    double *lever_y,
    double *lever_z,
    int *error);
```

The function **mb_lever** computes the effective lever arm correction (**lever_x*, **lever_y*, **lever_z*) that must be applied to sonar position because of the relative offsets between the sonar, the navigation antenna, and the attitude (VRU) sensor, given the current *vru_pitch* and *vru_roll*. This older, simpler lever arm calculation has

largely been superseded by the fuller sensor platform model implemented by **mb_platform_lever** below, but remains available for formats and programs that use it directly.

```
-----
int mb_beaudoin(
    int verbose,
    mb_3D_orientation tx_align,
    mb_3D_orientation tx_orientation,
    double tx_steer,
    mb_3D_orientation rx_align,
    mb_3D_orientation rx_orientation,
    double rx_steer,
    double reference_heading,
    double *beamAzimuth,
    double *beamDepression,
    int *error);
```

The function **mb_beaudoin** implements the algorithm of Beaudoin et al. for computing the true beam azimuth **beamAzimuth* and depression angle **beamDepression* of a multibeam sonar beam given the transmit and receive array mounting alignments (*tx_align*, *rx_align*), the vessel orientations at ping and receive time (*tx_orientation*, *rx_orientation*), the transmit and receive beam steering angles (*tx_steer*, *rx_steer*), and a *reference_heading*. This properly accounts for the different vessel attitudes at transmit and receive time on a moving platform.

```
-----
int mb_beaudoin_unrotate(
    int verbose,
    mb_3D_vector orig,
    mb_3D_orientation rotate,
    mb_3D_vector *final,
    int *error);
```

The function **mb_beaudoin_unrotate** rotates the vector *orig* by the inverse of the roll, pitch, and heading orientation *rotate*, returning the result in **final*. This is a supporting calculation used by **mb_beaudoin**.

```
-----
int mb_platform_init(
    int verbose,
    void **platform_ptr,
    int *error);
```

The function **mb_platform_init** allocates and initializes an empty sensor platform model, returning a pointer to it in **platform_ptr*. A platform model records the survey platform (ship, ROV, or AUV) identity together with a list of sensors, each with its position and attitude offsets relative to the platform's reference point, and is used throughout MBIO (see **mb_set_platform**, **mb_extract_platform**, and **mb_preprocess**) to convert raw sensor readings and geometry into properly located and oriented bathymetry, in place of the older, simpler **mb_lever** calculation. See also the **mbmakeplatform** manual page and the platform file format description therein.

```
-----
int mb_platform_setinfo(
    int verbose,
    void *platform_ptr,
```

```

int type,
char *name,
char *organization,
char *documentation_url,
double start_time_d,
double end_time_d,
int *error);

```

The function **mb_platform_setinfo** sets the identifying information of the platform pointed to by *platform_ptr*: its *type* (e.g. surface ship, ROV, AUV), *name*, *organization*, a *documentation_url*, and the time interval (*start_time_d* to *end_time_d*) over which this platform definition applies.

```

int mb_platform_add_sensor(
    int verbose,
    void *platform_ptr,
    int type,
    mb_longname model,
    mb_longname manufacturer,
    mb_longname serialnumber,
    int capability1,
    int capability2,
    int num_offsets,
    int num_time_latency,
    int *error);

```

The function **mb_platform_add_sensor** appends a new sensor record to the platform pointed to by *platform_ptr*, of the given *type* (e.g. sonar, INS, GPS), identified by *model*, *manufacturer*, and *serialnumber*, with capability flags *capability1* and *capability2* describing what the sensor measures, and space allocated for *num_offsets* position/attitude offset records (see **mb_platform_set_sensor_offset**) and *num_time_latency* time latency model samples (see **mb_platform_set_sensor_timelateness**). Sensors are indexed in the order added, starting from zero.

```

int mb_platform_set_sensor_offset(
    int verbose,
    void *platform_ptr,
    int isensor,
    int ioffset,
    double position_offset_x,
    double position_offset_y,
    double position_offset_z,
    double attitude_offset_azimuth,
    double attitude_offset_roll,
    double attitude_offset_pitch,
    int *error);

```

The function **mb_platform_set_sensor_offset** sets the *ioffset*th position offset (*position_offset_x*, *position_offset_y*, *position_offset_z*, in the platform's forward/starboard/down reference frame, meters) and attitude offset (*attitude_offset_azimuth*, *attitude_offset_roll*, *attitude_offset_pitch*, degrees) of sensor number *isensor* of the platform pointed to by *platform_ptr*, relative to the platform reference point.

```

int mb_platform_set_sensor_timelateness(
    int verbose,
    void *platform_ptr,
    int isensor,
    int time_latency_mode,
    double time_latency_static,
    int num_time_latency,
    double *time_latency_time_d,
    double *time_latency_value,
    int *error);

```

The function **mb_platform_set_sensor_timelateness** sets the time latency model for sensor number *isensor* of the platform pointed to by *platform_ptr*, in the same sense as **mb_apply_time_latency**: if *time_latency_mode* is **MB_SENSOR_TIME_LATENCY_STATIC** a constant *time_latency_static* offset is used, and if **MB_SENSOR_TIME_LATENCY_MODEL** the *num_time_latency*-point model given by *time_latency_time_d* and *time_latency_value* is used instead.

```

-----
int mb_platform_set_sensor_flipsign_heading(
    int verbose,
    void *platform_ptr,
    int isensor,
    int *error);
int mb_platform_set_sensor_flipsign_roll(
    int verbose,
    void *platform_ptr,
    int isensor,
    int *error);
int mb_platform_set_sensor_flipsign_pitch(
    int verbose,
    void *platform_ptr,
    int isensor,
    int *error);

```

The functions **mb_platform_set_sensor_flipsign_heading**, **mb_platform_set_sensor_flipsign_roll**, and **mb_platform_set_sensor_flipsign_pitch** each mark sensor number *isensor* of the platform pointed to by *platform_ptr* as having its heading, roll, or pitch sign convention reversed relative to the **MB-System** standard, so that **mb_platform_apply_flipsign_heading** and **mb_platform_apply_flipsign_attitude** will correct it.

```

-----
int mb_platform_set_source_sensor(
    int verbose,
    void *platform_ptr,
    int source_type,
    int sensor,
    int *error);

```

The function **mb_platform_set_source_sensor** designates sensor number *sensor* of the platform pointed to by *platform_ptr* as the source of a particular kind of data (*source_type* - e.g. navigation, heading, roll/pitch, heave, or sonar depth), so that functions such as **mb_platform_orientation** know which sensor's offsets to apply.

```
int mb_platform_deall(
    int verbose,
    void **platform_ptr,
    int *error);
```

The function **mb_platform_deall** deallocates a platform model previously allocated by **mb_platform_init** or **mb_platform_read** and pointed to by *platform_ptr*.

```
-----
int mb_platform_read(
    int verbose,
    char *platform_file,
    void **platform_ptr,
    int *error);
```

The function **mb_platform_read** reads a platform definition from the file *platform_file* (as written by **mb_platform_write** or the **mbmakeplatform** program) and returns a newly allocated platform model in *platform_ptr*.

```
-----
int mb_platform_write(
    int verbose,
    char *platform_file,
    void *platform_ptr,
    int *error);
```

The function **mb_platform_write** writes the platform model pointed to by *platform_ptr* to the file *platform_file*, in the format read by **mb_platform_read**.

```
-----
int mb_platform_lever(
    int verbose,
    void *platform_ptr,
    int targetsensor,
    int targetsensoroffset,
    double heading,
    double roll,
    double pitch,
    double *lever_x,
    double *lever_y,
    double *lever_z,
    int *error);
```

The function **mb_platform_lever** computes the lever arm correction (**lever_x*, **lever_y*, **lever_z*, in the platform reference frame) between the platform's navigation reference point and offset number *targetsensoroffset* of sensor number *targetsensor* of the platform pointed to by *platform_ptr*, given the platform's current *heading*, *roll*, and *pitch*. This generalizes the older **mb_lever** calculation to the full multi-sensor platform model.

```
-----
int mb_platform_position(
    int verbose,
    void *platform_ptr,
    int targetsensor,
```

```

    int targetsensoroffset,
    double navlon,
    double navlat,
    double sensordepth,
    double heading,
    double roll,
    double pitch,
    double *targetlon,
    double *targetlat,
    double *targetz,
    int *error);

```

The function **mb_platform_position** translates a navigation fix (*navlon*, *navlat*, *sensordepth*) obtained at the platform's navigation reference point, together with the current *heading*, *roll*, and *pitch*, into the corresponding position (**targetlon*, **targetlat*, **targetz*) of offset number *targetsensoroffset* of sensor number *targetsensor* of the platform pointed to by *platform_ptr*, applying the lever arm computed by **mb_platform_lever**.

```

-----
int mb_platform_position_offset(
    int verbose,
    void *platform_ptr,
    int targetsensor,
    int targetsensoroffset,
    double *target_x_offset,
    double *target_y_offset,
    double *target_z_offset,
    int *error);

```

The function **mb_platform_position_offset** returns just the position offset (**target_x_offset*, **target_y_offset*, **target_z_offset*, in the platform reference frame) of offset number *targetsensoroffset* of sensor number *targetsensor* of the platform pointed to by *platform_ptr*, without applying any attitude rotation.

```

-----
int mb_platform_apply_flipsign_attitude(
    int verbose,
    void *platform_ptr,
    double *roll,
    double *pitch,
    int *error);

```

The function **mb_platform_apply_flipsign_attitude** negates **roll* and/or **pitch* in place if the corresponding attitude source sensor of the platform pointed to by *platform_ptr* has been marked with **mb_platform_set_sensor_flipsign_roll** or **mb_platform_set_sensor_flipsign_pitch**.

```

-----
int mb_platform_apply_flipsign_heading(
    int verbose,
    void *platform_ptr,
    double *heading,
    int *error);

```

The function **mb_platform_apply_flipsign_heading** corrects **heading* in place if the heading source sensor of the platform pointed to by *platform_ptr* has been marked with

mb_platform_set_sensor_flipsign_heading.

```

-----
int mb_platform_orientation(
    int verbose,
    void *platform_ptr,
    double heading,
    double roll,
    double pitch,
    double *platform_heading,
    double *platform_roll,
    double *platform_pitch,
    int *error);

```

The function **mb_platform_orientation** converts a raw *heading*, *roll*, and *pitch* reading from the platform's designated attitude and heading source sensors (see **mb_platform_set_source_sensor**) into the equivalent platform reference frame orientation (**platform_heading*, **platform_roll*, **platform_pitch*), correcting for that sensor's mounting angle offsets.

```

-----
int mb_platform_orientation_offset(
    int verbose,
    void *platform_ptr,
    int targetsensor,
    int targetsensoroffset,
    double *target_hdg_offset,
    double *target_roll_offset,
    double *target_pitch_offset,
    int *error);

```

The function **mb_platform_orientation_offset** returns just the mounting angle offsets (**target_hdg_offset*, **target_roll_offset*, **target_pitch_offset*) of offset number *targetsensoroffset* of sensor number *targetsensor* of the platform pointed to by *platform_ptr*.

```

-----
int mb_platform_orientation_target(
    int verbose,
    void *platform_ptr,
    int targetsensor,
    int targetsensoroffset,
    double heading,
    double roll,
    double pitch,
    double *target_heading,
    double *target_roll,
    double *target_pitch,
    int *error);

```

The function **mb_platform_orientation_target** converts the platform reference frame orientation (*heading*, *roll*, *pitch*) into the corresponding orientation (**target_heading*, **target_roll*, **target_pitch*) of offset number *targetsensoroffset* of sensor number *targetsensor* of the platform pointed to by *platform_ptr*, applying that offset's mounting angles. This is the inverse sense of **mb_platform_orientation**.

```

-----
int mb_platform_copy(
    int verbose,
    void *platform_ptr,
    void *platform_copy_ptr,
    int *error);

```

The function **mb_platform_copy** copies the platform model pointed to by *platform_ptr* into the already-allocated platform structure pointed to by *platform_copy_ptr*.

```

-----
int mb_platform_print(
    int verbose,
    void *platform_ptr,
    int *error);

```

The function **mb_platform_print** prints, to standard error, a human-readable listing of the platform model pointed to by *platform_ptr*, including every sensor and its offsets. This is invoked automatically by many of the **mb_platform_*** functions above when *verbose* is 2 or greater.

```

-----
void mb_platform_math_matrix_times_vector_3x1(
    double *A,
    double *b,
    double *Ab);
void mb_platform_math_matrix_times_matrix_3x3(
    double *A,
    double *B,
    double *AB);
void mb_platform_math_matrix_transpose_3x3(
    double *R,
    double *R_T);

```

The functions **mb_platform_math_matrix_times_vector_3x1**, **mb_platform_math_matrix_times_matrix_3x3**, and **mb_platform_math_matrix_transpose_3x3** are basic 3x3 matrix and 3-vector arithmetic helpers (matrix-vector product $*Ab = *A$ times $*b$, matrix product $*AB = *A$ times $*B$, and transpose $*R_T$ of $*R$) used by the platform orientation calculations below.

```

-----
void mb_platform_math_rph2rot(
    double *rph,
    double *R);
void mb_platform_math_rot2rph(
    double *R,
    double *rph);

```

The function **mb_platform_math_rph2rot** converts a roll/pitch/heading triple *rph* into the equivalent 3x3 rotation matrix *R*; **mb_platform_math_rot2rph** performs the inverse conversion.

```

-----
int mb_platform_math_attitude_offset(
    int verbose,
    double target_offset_roll,

```



```

double target_offset_pitch,
double target_offset_heading,
double source_offset_roll,
double source_offset_pitch,
double source_offset_heading,
double *target2source_offset_roll,
double *target2source_offset_pitch,
double *target2source_offset_heading,
int *error);

```

The function **mb_platform_math_attitude_offset** computes the relative mounting angle offset (**target2source_offset_roll*, **target2source_offset_pitch*, **target2source_offset_heading*) between a target sensor mounted with offset (*target_offset_roll*, *target_offset_pitch*, *target_offset_heading*) and a source sensor mounted with offset (*source_offset_roll*, *source_offset_pitch*, *source_offset_heading*), both relative to the platform frame. This is a supporting calculation used by **mb_platform_orientation_offset** and related functions.

```

-----
int mb_platform_math_attitude_platform(
    int verbose,
    double nav_attitude_roll,
    double nav_attitude_pitch,
    double nav_attitude_heading,
    double attitude_offset_roll,
    double attitude_offset_pitch,
    double attitude_offset_heading,
    double *platform_roll,
    double *platform_pitch,
    double *platform_heading,
    int *error);

```

The function **mb_platform_math_attitude_platform** converts an attitude reading (*nav_attitude_roll*, *nav_attitude_pitch*, *nav_attitude_heading*) made by a sensor mounted with offset (*attitude_offset_roll*, *attitude_offset_pitch*, *attitude_offset_heading*) into the platform frame attitude (**platform_roll*, **platform_pitch*, **platform_heading*). This is a supporting calculation used by **mb_platform_orientation**.

```

-----
int mb_platform_math_attitude_target(
    int verbose,
    double source_attitude_roll,
    double source_attitude_pitch,
    double source_attitude_heading,
    double target_offset_to_source_roll,
    double target_offset_to_source_pitch,
    double target_offset_to_source_heading,
    double *target_roll,
    double *target_pitch,
    double *target_heading,
    int *error);

```

The function **mb_platform_math_attitude_target** converts an attitude (*source_attitude_roll*, *source_attitude_pitch*, *source_attitude_heading*) known in the source sensor's frame into the corresponding attitude (**target_roll*, **target_pitch*, **target_heading*) in the target sensor's frame, given the mounting offset of the target relative to the source (*target_offset_to_source_roll*, *target_offset_to_source_pitch*,

target_offset_to_source_heading). This is a supporting calculation used by **mb_platform_orientation_target**.

```
-----
int mb_platform_math_attitude_offset_corrected_by_nav(
    int verbose,
    double prev_attitude_roll,
    double prev_attitude_pitch,
    double prev_attitude_heading,
    double target_offset_to_source_roll,
    double target_offset_to_source_pitch,
    double target_offset_to_source_heading,
    double new_attitude_roll,
    double new_attitude_pitch,
    double new_attitude_heading,
    double *corrected_offset_roll,
    double *corrected_offset_pitch,
    double *corrected_offset_heading,
    int *error);
```

The function **mb_platform_math_attitude_offset_corrected_by_nav** refines a previously estimated mounting angle offset (*target_offset_to_source_roll*, *target_offset_to_source_pitch*, *target_offset_to_source_heading*) using a change in navigation attitude from (*prev_attitude_roll*, *prev_attitude_pitch*, *prev_attitude_heading*) to (*new_attitude_roll*, *new_attitude_pitch*, *new_attitude_heading*), returning the corrected offset in (**corrected_offset_roll*, **corrected_offset_pitch*, **corrected_offset_heading*). This is a supporting calculation used by patch-test style sensor alignment calibration tools.

```
-----
int mb_platform_math_attitude_rotate_beam(
    int verbose,
    double beam_acrosstrack,
    double beam_alongtrack,
    double beam_bath,
    double attitude_roll,
    double attitude_pitch,
    double attitude_heading,
    double *newbeam_easting,
    double *newbeam_northing,
    double *newbeam_bath,
    int *error);
```

The function **mb_platform_math_attitude_rotate_beam** rotates a single bathymetry beam position (*beam_acrosstrack*, *beam_alongtrack*, *beam_bath*) by the attitude (*attitude_roll*, *attitude_pitch*, *attitude_heading*) into georeferenced easting, northing, and depth (**newbeam_easting*, **newbeam_northing*, **newbeam_bath*).

```
-----
int mb_double_compare(
    const void *a,
    const void *b);
```

The function **mb_double_compare** is used with the **qsort** function to sort arrays of *double* values. This function returns 1 if *a > b* and -1 if *a <= b*.

```
-----
int mb_int_compare(
    const void *a,
    const void *b);
```

The function **mb_int_compare** is used with the **qsort** function to sort arrays of *int* values. This function returns 1 if $a > b$ and -1 if $a \leq b$.

```
-----
int mb_edit_compare(
    const void *a,
    const void *b);
```

The function **mb_edit_compare** is used with the **qsort** function to sort arrays of *struct mb_edit_struct* beam edit event records (as used in edit save files - see the **mbedit** and **mbclean** manual pages) by time and beam number. Two edit events whose times differ by less than a small tolerance are considered simultaneous and are then ordered by beam number.

```
-----
int mb_edit_compare_coarse(
    const void *a,
    const void *b);
```

The function **mb_edit_compare_coarse** is used with the **qsort** function to sort arrays of *struct mb_edit_struct* beam edit event records in the same manner as **mb_edit_compare**, except that a coarser (larger) time tolerance is used in judging two edit events to be simultaneous.

```
-----
int mb_absorption(
    int verbose,
    double frequency,
    double temperature,
    double salinity,
    double depth,
    double ph,
    double soundspeed,
    double *absorption,
    int *error);
```

The function **mb_absorption** calculates the sound absorption coefficient **absorption* (dB/km) of sea water for a sonar signal of *frequency* (kHz) given the water *temperature* (degrees C), *salinity* (per mil), *depth* (meters), *ph*, and *soundspeed* (m/sec), following the standard Francois-Garrison absorption model. This is used, for example, by **mbbackangle** and **mbsvpselect** when correcting backscatter for water column transmission losses.

```
-----
int mb_potential_temperature(
    int verbose,
    double temperature,
    double salinity,
    double pressure,
    double *potential_temperature,
    int *error);
```

The function **mb_potential_temperature** calculates the potential temperature **potential_temperature* (degrees C) of sea water given its in-situ *temperature* (degrees C), *salinity* (per mil), and *pressure* (dbar).

```
-----
int mb_seabird_density(
    int verbose,
    double salinity,
    double temperature,
    double pressure,
    double *density,
    int *error);
```

The function **mb_seabird_density** calculates the density **density* (kg/m³) of sea water given its *salinity* (per mil), *temperature* (degrees C), and *pressure* (dbar), following the algorithms used by Sea-Bird Electronics CTD processing software.

```
-----
int mb_seabird_depth(
    int verbose,
    double pressure,
    double latitude,
    double *depth,
    int *error);
```

The function **mb_seabird_depth** calculates the depth **depth* (meters) below the sea surface corresponding to a CTD *pressure* (dbar) reading at the given *latitude*, following the algorithms used by Sea-Bird Electronics CTD processing software.

```
-----
int mb_seabird_salinity(
    int verbose,
    double conductivity,
    double temperature,
    double pressure,
    double *salinity,
    int *error);
```

The function **mb_seabird_salinity** calculates the salinity **salinity* (per mil) of sea water given its *conductivity*, *temperature* (degrees C), and *pressure* (dbar), following the algorithms used by Sea-Bird Electronics CTD processing software.

```
-----
int mb_seabird_soundspeed(
    int verbose,
    int algorithm,
    double salinity,
    double temperature,
    double pressure,
    double *soundspeed,
    int *error);
```

The function **mb_seabird_soundspeed** calculates the sound speed **soundspeed* (m/sec) of sea water given its *salinity* (per mil), *temperature* (degrees C), and *pressure* (dbar), using the sound speed formula selected by

algorithm, following the algorithms used by Sea-Bird Electronics CTD processing software.

```
-----
int mb_rt_init(
    int verbose,
    int number_node,
    double *depth,
    double *velocity,
    void **modelptr,
    int *error);
```

The function **mb_rt_init** initializes a layered water sound velocity model for ray tracing, given *number_node* depth/velocity pairs in the arrays *depth* and *velocity*, and returns a pointer to the model in **modelptr* for use by **mb_rt**.

```
-----
int mb_rt_deall(
    int verbose,
    void **modelptr,
    int *error);
```

The function **mb_rt_deall** deallocates a velocity model previously initialized by **mb_rt_init** and pointed to by **modelptr*.

```
-----
int mb_rt(
    int verbose,
    void *modelptr,
    double source_depth,
    double source_angle,
    double end_time,
    int ssv_mode,
    double surface_vel,
    double null_angle,
    int nplot_max,
    int *nplot,
    double *xplot,
    double *zplot,
    double *tplot,
    double *x,
    double *z,
    double *travel_time,
    int *ray_stat,
    int *error);
```

The function **mb_rt** traces a single ray, launched from *source_depth* at takeoff angle *source_angle*, through the layered velocity model *modelptr* (initialized by **mb_rt_init**) until either the elapsed travel time reaches *end_time* or the ray exits the model (surfacing, in which case *ssv_mode* and *surface_vel* control how the ray reflects off the sea surface, and rays arriving within *null_angle* of vertical are treated specially). The ray path is returned as up to *nplot_max* plotting points in *xplot/zplot/tplot* (with the actual number of points returned in **nplot*), and the ray's final horizontal range **x*, depth **z*, and elapsed **travel_time* are returned along with a **ray_stat* code describing how the ray terminated. Internally **mb_rt** calls **mb_rt_get_depth** and the family of **mb_rt_quad1**, **mb_rt_quad2**, **mb_rt_quad3**, **mb_rt_quad4**, **mb_rt_circular**, **mb_rt_plot_circular**,

mb_rt_line, and **mb_rt_vertical** functions below to trace the ray through each layer of the model according to the local velocity gradient and ray geometry; these are implementation details of the ray tracing algorithm and are not normally called directly by application programs.

```
-----
int mb_rt_get_depth(
    int verbose,
    void *modelptr,
    double beta,
    int dir_sign,
    int turn_sign,
    double *depth,
    int *error);
```

The function **mb_rt_get_depth** returns the depth **depth* in the current layer of the velocity model *modelptr* corresponding to ray parameter *beta*, given the ray's current vertical travel direction *dir_sign* and turning state *turn_sign*. This is a supporting calculation used by **mb_rt**.

```
-----
int mb_rt_quad1(
    int verbose,
    void *modelptr,
    int *error);
int mb_rt_quad2(
    int verbose,
    void *modelptr,
    int *error);
int mb_rt_quad3(
    int verbose,
    void *modelptr,
    int *error);
int mb_rt_quad4(
    int verbose,
    void *modelptr,
    int *error);
```

The functions **mb_rt_quad1**, **mb_rt_quad2**, **mb_rt_quad3**, and **mb_rt_quad4** each advance the ray being traced through the velocity model *modelptr* across one quadrant of curvature, handling the four combinations of upward or downward ray travel with increasing or decreasing velocity gradient. These are supporting calculations used internally by **mb_rt** and are not normally called directly.

```
-----
int mb_rt_circular(
    int verbose,
    void *modelptr,
    int *error);
```

The function **mb_rt_circular** advances the ray being traced through the velocity model *modelptr* along a circular arc appropriate to a layer of constant velocity gradient. This is a supporting calculation used internally by **mb_rt**.

```
-----
int mb_rt_plot_circular(
```

```

    int verbose,
    void *modelptr,
    int *error);

```

The function **mb_rt_plot_circular** generates the plotting points recording the circular ray path segment computed by **mb_rt_circular**. This is a supporting calculation used internally by **mb_rt**.

```

-----
int mb_rt_line(
    int verbose,
    void *modelptr,
    int *error);

```

The function **mb_rt_line** advances the ray being traced through the velocity model *modelptr* along a straight line segment appropriate to a layer of constant velocity. This is a supporting calculation used internally by **mb_rt**.

```

-----
int mb_rt_vertical(
    int verbose,
    void *modelptr,
    int *error);

```

The function **mb_rt_vertical** advances the ray being traced through the velocity model *modelptr* for the special case of a vertically traveling ray. This is a supporting calculation used internally by **mb_rt**.

COORDINATE SYSTEMS USED IN MB-SYSTEM

I. Introduction

The coordinate systems described below are used within **MB-System** for calculations involving the location in space of depth, amplitude, or sidescan data. In all cases the origin of the coordinate system is at the center of the sonar transducers.

II. Cartesian Coordinates

The cartesian coordinate system used in **MB-System** is a bit odd because it is left-handed, as opposed to the right-handed x-y-z space conventionally used in most circumstances. With respect to the sonar (or the ship on which the sonar is mounted), the x-axis is athwartships with positive to starboard (to the right if facing forward), the y-axis is fore-aft with positive forward, and the z-axis is positive down.

III. Spherical Coordinates

There are two non-traditional spherical coordinate systems used in **MB-System**. The first, referred to here as takeoff angle coordinates, is useful for raytracing. The second, referred to here as roll-pitch coordinates, is useful for taking account of corrections to roll and pitch angles.

III.1. Takeoff Angle Coordinates

The three parameters are r, theta, and phi, where r is the distance from the origin, theta is the angle from vertical down (that is, from the positive z-axis), and phi is the angle from across-track (the positive x-axis) in the x-y plane. Note that theta is always positive; the direction in the x-y plane is given by phi. Raytracing is simple in these coordinates because the ray takeoff angle is just theta. However, applying roll or pitch corrections is complicated because roll and pitch have components in both theta and phi.

```

0 <= theta <= PI/2
-PI/2 <= phi <= 3*PI/2

```

$x = r \sin(\theta) \cos(\phi)$
 $y = r \sin(\theta) \sin(\phi)$
 $z = r \cos(\theta)$

$\theta = 0$ ---> vertical, along positive z-axis
 $\theta = \pi/2$ ---> horizontal, in x-y plane
 $\phi = -\pi/2$ ---> aft, in y-z plane with y negative
 $\phi = 0$ ---> port, in x-z plane with x positive
 $\phi = \pi/2$ ---> forward, in y-z plane with y positive
 $\phi = \pi$ ---> starboard, in x-z plane with x negative
 $\phi = 3\pi/2$ ---> aft, in y-z plane with y negative

III.2. Roll-Pitch Coordinates

The three parameters are r, alpha, and beta, where r is the distance from the origin, alpha is the angle forward (effectively pitch angle), and beta is the angle from horizontal in the x-z plane (effectively roll angle). Applying a roll or pitch correction is simple in these coordinates because pitch is just alpha and roll is just beta. However, raytracing is complicated because deflection from vertical has components in both alpha and beta.

$-\pi/2 \leq \alpha \leq \pi/2$
 $0 \leq \beta \leq \pi$

$x = r \cos(\alpha) \cos(\beta)$
 $y = r \sin(\alpha)$
 $z = r \cos(\alpha) \sin(\beta)$

$\alpha = -\pi/2$ ---> horizontal, in x-y plane with y negative
 $\alpha = 0$ ---> ship level, zero pitch, in x-z plane
 $\alpha = \pi/2$ ---> horizontal, in x-y plane with y positive
 $\beta = 0$ ---> starboard, along positive x-axis
 $\beta = \pi/2$ ---> in y-z plane rotated by alpha
 $\beta = \pi$ ---> port, along negative x-axis

IV. SeaBeam Coordinates

The per-beam parameters in the SB2100 data format include angle-from-vertical and angle-forward. Angle-from-vertical is the same as theta except that it is signed based on the across-track direction (positive to starboard, negative to port). The angle-forward values are also defined slightly differently from phi, in that angle-forward is signed differently on the port and starboard sides. The SeaBeam 2100 External Interface Specifications document includes both discussion and figures illustrating the angle-forward value. To summarize:

Port:

$\theta = \text{absolute value of angle-from-vertical}$

$-\pi/2 \leq \phi \leq \pi/2$
 is equivalent to
 $-\pi/2 \leq \text{angle-forward} \leq \pi/2$

$\phi = -\pi/2$ ---> angle-forward = $-\pi/2$ (aft)
 $\phi = 0$ ---> angle-forward = 0 (starboard)
 $\phi = \pi/2$ ---> angle-forward = $\pi/2$ (forward)

Starboard:

$\theta = \text{angle-from-vertical}$

$\pi/2 \leq \phi \leq 3\pi/2$

is equivalent to

$-\pi/2 \leq \text{angle-forward} \leq \pi/2$

$\phi = \pi/2 \rightarrow \text{angle-forward} = -\pi/2$ (forward)

$\phi = \pi \rightarrow \text{angle-forward} = 0$ (port)

$\phi = 3\pi/2 \rightarrow \text{angle-forward} = \pi/2$ (aft)

V. Usage of Coordinate Systems in **MB-System**

Some sonar data formats provide angle values along with travel times. The angles are converted to takeoff-angle coordinates regardless of the storage form of the particular data format. Currently, most data formats do not contain an alongtrack component to the position values; in these cases the conversion is trivial since $\phi = \beta = 0$ and $\theta = \alpha$. The angle and travel time values can be accessed using the **MBIO** function **mb_times**. All angle values passed by **MB-System** functions are in degrees rather than radians.

The programs **mbbath** and **mbvelocitytool** use angles in take-off angle coordinates to do the raytracing. If roll and/or pitch corrections are to be made, the angles are converted to roll-pitch coordinates, corrected, and then converted back prior to raytracing.

BEAM FLAGS USED IN MB-SYSTEM

MB-System uses arrays of 1-byte "beamflag" values to indicate beam data quality. Each beamflag value is actually an eight bit mask allowing fairly complicated information to be stored regarding each bathymetry value. In particular, beams may be flagged as bad, they may be selected as being of special interest, and one or more reasons for flagging or selection may be indicated. This scheme is very similar to the convention used in the HMPS hydrographic data processing package and the SAIC Hydrobat package. The beam selection mechanism is not currently used by any **MB-System** programs.

The flag and select bits:

xxxxxx00 => This beam is neither flagged nor selected.

xxxxxx01 => This beam is flagged as bad and should be ignored.

xxxxxx10 => This beam has been selected.

Flagging modes:

00000001 => Flagged because no detection was made by the sonar.

xxxxx101 => Flagged by manual editing.

xxxx1x01 => Flagged by automatic filter.

xxx1xx01 => Flagged by automatic filter in the current program.

xx1xxx01 => Flagged because this is a secondary bottom pick.

x1xxxx01 => Flagged because this is an interpolated rather than observed sounding.

1xxxxx01 => Flagged as unreliable using original quality values from sonar.

(The bits at xxx1xx01, xx1xxx01, and x1xxxx01 originally meant, respectively, that the uncertainty exceeded 1 X or 2 X the IHO standard, or that the footprint was too large; those original meanings are deprecated.)

Selection modes:

00000010 => Selected, no reason specified.

xxxxx110 => Selected as least depth.
xxxx1x10 => Selected as average depth.
xxx1xx10 => Selected as maximum depth.
xx1xxx10 => Selected as location of sidescan contact.
x1xxxx10 => Selected, spare.
1xxxxx10 => Selected, spare.

SEE ALSO

mbsystem(l), **mbformat(l)**

BUGS

What could go wrong in a mere 374,852 lines of C code?
Let us know...